

UNIVERSIDAD DE MÁLAGA
ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA DE TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

MIGRACIÓN DE UN CANAL DE VÍDEO EN
TIEMPO REAL DESDE UNA FPGA SOC A UNA
FPGA DISCRETA DE BAJO COSTE

GRADO EN INGENIERÍA DE
TECNOLOGÍAS DE LA TELECOMUNICACIÓN

ALFONSO SUÁREZ GASULL
MÁLAGA, 2026

Migración de un canal de vídeo en tiempo real desde una FPGA SoC a una FPGA discreta de bajo coste

Autor: Alfonso Suárez Gasull

Tutor: Martín González García

Cotutor: Juan Antonio Rodríguez Fernández

Departamento: Tecnología Electrónica

Titulación: Grado en Ingeniería de Tecnologías de Telecomunicación

Palabras clave: Matriz de Puertas Lógicas Programable (FPGA), canal de vídeo en tiempo real, migración hardware, MicroBlaze, Síntesis de Alto Nivel (HLS), procesamiento de vídeo, protocolo AXI

Resumen

Este trabajo aborda la migración de un canal de vídeo en tiempo real, originalmente implementado sobre una plataforma SoC Zynq-7000, a una FPGA discreta de bajo coste basada en Spartan-7. El sistema de referencia captura imágenes desde un sensor CMOS, las almacena en memoria externa y genera una salida HDMI, apoyándose en los recursos integrados de la plataforma. El objetivo es reproducir este canal sobre un dispositivo sin procesador dedicado, sustituyendo en lógica programable las funciones que antes resolvía el sistema de procesamiento. Para ello, se emplea un procesador *soft-core*, un controlador de memoria DDR y los periféricos necesarios, integrados en una arquitectura de interconexión propia. El sistema se amplía además con un subsistema de monitorización y un módulo de procesamiento de vídeo diseñado mediante síntesis de alto nivel. Validado y caracterizado con medidas reales, el resultado confirma la viabilidad de la migración: reproduce la funcionalidad de la referencia con un consumo similar y una ocupación de recursos lógicos equivalente sobre una plataforma sensiblemente más económica, siendo las decisiones de diseño determinantes para su portabilidad a dispositivos basados en FPGA de menor capacidad.

Migration of a real-time video channel from a SoC FPGA to a low-cost discrete FPGA

Author: Alfonso Suárez Gasull

Supervisor: Martín González García

Co-supervisor: Juan Antonio Rodríguez Fernández

Department: Electronic Technology

Degree: Grado en Ingeniería de Tecnologías de Telecomunicación

Keywords: Field Programmable Gate Array (FPGA), real-time video channel, hardware migration, MicroBlaze, High-Level Synthesis (HLS), video processing, AXI protocol

Abstract

This work addresses the migration of a real-time video channel, originally implemented on a Zynq-7000 SoC platform, to a low-cost discrete FPGA based on the Spartan-7 device. The reference system captures images from a CMOS sensor, stores them in external memory and generates an HDMI video output, relying on the platform's integrated resources. The goal is to reproduce this channel on a device without a dedicated processor, replacing in the programmable logic the functions previously handled by the processing system. To this end, a soft-core processor, a DDR memory controller and the required peripherals are integrated into a custom interconnect. Furthermore, the system is extended with a monitoring subsystem and a video processing module designed through high-level synthesis. Validated and characterized with real measurements, the result confirms the feasibility of the system migration: it reproduces the functionality of the reference project with power consumption of the same order and an equivalent use of logical resources on a significantly more affordable platform, with the design decisions proving decisive for its portability to lower capacity FPGA-based devices.

Agradecimientos

En primer lugar, desearía agradecer a mis tutores, Juan Antonio y Martín, por haberme dado la posibilidad de realizar este tan interesante proyecto, proporcionarme todo el material necesario, guiarme y aconsejarme en momentos que buena falta me han hecho y, especialmente, por transmitirme la pasión que actualmente siento por la microelectrónica.

También me gustaría agradecer a mis padres por haber estado siempre ahí para mí, por su paciencia, sus ánimos, cariño y comprensión durante toda mi vida. Me hacen sentir muy afortunado de la crianza que he recibido. Querría agradecer especialmente a mi padre, por haberme echado un cable con las soldaduras.

Por último, me gustaría agradecer a mi gran apoyo durante estos últimos meses. Alguien que me ha hecho siempre creer en mí, en los buenos y en los malos momentos. Muchas gracias, Sara. Si ha habido veces que he estado a punto de perder la esperanza, tú me las has devuelto siempre. Soy muy afortunado de contar contigo cada día.

Contenido

Capítulo 1. Introducción.....	1
1.1. Motivación.....	1
1.2. Objetivos específicos.....	2
1.3. Metodología.....	2
1.4. Organización de la memoria.....	3
Capítulo 2. Estado del arte	4
2.1. Procesado de vídeo sobre FPGA.....	4
2.2. Protocolos AXI.....	6
2.3. Plataformas FPGA SoC y FPGA discretas	7
2.4. Procesadores soft-core y MicroBlaze.....	8
2.5. Síntesis de Alto Nivel (HLS).....	8
Capítulo 3. Canal de vídeo de referencia basado en FPGA SoC	10
3.1. Descripción general del sistema de referencia.....	10
3.2. Cadena de procesamiento de vídeo (PL).....	11
3.3. Funciones del Sistema de Procesamiento.....	13
3.4. Consideraciones para la migración del sistema.....	14
Capítulo 4. Migración del sistema a una FPGA discreta	16
4.1. Plataforma objetivo: Arty S7-50.....	16
4.2. Arquitectura general del sistema migrado.....	17
4.3. Plano de Control.....	19
4.3.1. Sistema de control basado en MicroBlaze.....	19
4.3.2. Interfaz de memoria DDR3L (MIG).....	21
4.3.3. Periféricos de comunicación	23
4.4. Infraestructuras de interconexión y sincronización.....	23

4.5. Adaptación de salida HDMI.....	26
4.6. Integración y validación del sistema migrado	28
Capítulo 5. Ampliación del sistema de vídeo	31
5.1. Objetivos de la ampliación	31
5.2. Sistema de monitorización.....	32
5.2.1. Medida del ancho de banda de memoria	33
5.2.2. Medida del tiempo de ejecución software	33
5.3. Módulo de procesamiento y generación de vídeo mediante HLS	34
5.3.1. Arquitectura e integración del módulo	34
5.3.2. Modos de funcionamiento.....	36
5.3.3. Superposición de métricas en pantalla (OSD)	39
5.3.4. Sincronización del OSD con el flujo de vídeo	41
5.4. Arquitectura del sistema de vídeo completo.....	42
5.5. Aplicación software de control final.....	43
Capítulo 6. Resultados y validación.....	46
6.1. Validación funcional.....	46
6.2. Ancho de banda de memoria.....	52
6.3. Utilización de recursos del sistema migrado.....	53
6.3.1. Decisiones de diseño de la migración.....	55
6.4. Coste del sistema ampliado	57
6.5. Potencia	59
6.6. Comparación con el sistema de referencia.....	62
6.7. Limitaciones	68
Capítulo 7. Conclusiones	69
7.1. Líneas futuras	70
Referencias	72

Apéndice A. Software adjunto y repositorio del proyecto 75

Lista de Figuras

FIGURA 2.1 EJEMPLO DE APLICACIÓN DE FRAME BUFFERS ARBITRADOS.	5
FIGURA 2.2 EJEMPLO DE PIPELINE APLICADO A CADENA DE VÍDEO.....	6
FIGURA 3.1 ARQUITECTURA GENERAL DEL SISTEMA DE REFERENCIA.	11
FIGURA 3.2 CADENA DE PROCESADO DE VÍDEO DEL SISTEMA DE REFERENCIA.	11
FIGURA 3.3 FLUJOGRAMA DE INICIALIZACIÓN DEL PROYECTO DE REFERENCIA.	13
FIGURA 4.1. PLATAFORMA ARTY S7-50 Y COMPONENTES EMPLEADOS EN EL PROYECTO. [9].....	17
FIGURA 4.2. ARQUITECTURA DEL SISTEMA DE VÍDEO MIGRADO A LA ARTY S7-50.....	18
FIGURA 4.3. IMPLEMENTACIÓN DEL PROCESADOR MICROBLAZE.	20
FIGURA 4.4 REDES DE INTERCONEXIÓN DE CONTROL (COLOR AMARILLO) Y DE MEMORIA (COLOR NARANJA).....	24
FIGURA 4.5. DESCRIPCIÓN DE PINES DE UN CONECTOR HDMI (VISTA TRASERA).[13].....	27
FIGURA 4.6. ASIGNACIÓN DE CANALES TMDS A LOS PINES DEL PMOD JB.....	27
FIGURA 4.7. IMPLEMENTACIÓN EN VIVADO DEL SISTEMA DE VÍDEO MIGRADO.	28
FIGURA 4.8 MONTAJE DE PRUEBAS. ARTY S7-50, CÁMARA Y SALIDA HDMI ADAPTADA.	29
FIGURA 4.9. IMAGEN CAPTURADA Y VISUALIZADA EN TIEMPO REAL.	30
FIGURA 5.1 PERIFÉRICOS DE MONITORIZACIÓN EN LA RED DE CONTROL (EN NARANJA).....	32
FIGURA 5.2 SONDA DEL APM SOBRE LA INTERFAZ AXI4 DEL MIG (EN NARANJA).....	33
FIGURA 5.3 INTEGRACIÓN DEL MÓDULO HLS EN EL FLUJO DE VÍDEO (CONEXIONES EN NARANJA).	35
FIGURA 5.4 DIRECTIVAS HLS DE INTERFACES Y RELOJ DE CONTROL.	36
FIGURA 5.5 MECANISMO DE SALVAGUARDA.	39
FIGURA 5.6 ESQUEMA DE REPRESENTACIÓN DEL OSD EN UN FOTOGRAMA.	40
FIGURA 5.7 INTERACCIÓN HARDWARE-SOFTWARE PARA LA REPRESENTACIÓN DEL OSD.	40

FIGURA 5.8 FLUJO DE LA ETAPA DE RESINCRONIZACIÓN DEL OSD EN LOS MODOS DE PROCESADO.	42
FIGURA 5.9 SISTEMA DE VÍDEO COMPLETO.	42
FIGURA 5.10 DIAGRAMA DE FLUJO DE LA APLICACIÓN SOFTWARE FINAL.	43
FIGURA 6.1 MENSAJES DE INICIALIZACIÓN DEL SISTEMA POR TERMINAL SERIE.	47
FIGURA 6.2 SALIDA DE VÍDEO EN MODO DE PASO DIRECTO.	48
FIGURA 6.3 SALIDA DE VÍDEO EN MODO DE ESCALA DE GRISES.	48
FIGURA 6.4 MODO DE GENERACIÓN: PATRÓN DE BARRAS DE COLOR.	49
FIGURA 6.5 MODO DE GENERACIÓN: PATRÓN DINÁMICO	49
FIGURA 6.6 OSD ALINEADO ANTES DE LA CONMUTACIÓN DE MODO.	50
FIGURA 6.7 OSD ALINEADO TRAS LA CONMUTACIÓN DE MODO.	50
FIGURA 6.8 PATRÓN DE AUSENCIA DE SEÑAL.	51
FIGURA 6.9 MEDIDA DE ANCHO DE BANDA EN CASO DE PROCESADO DE VÍDEO.	51
FIGURA 6.10 TIEMPO DE EJECUCIÓN DE MICROBLAZE MEDIDO.	52
FIGURA 6.11 OCUPACIÓN DE CADA SISTEMA RESPECTO DEL TOTAL DE CADA PLATAFORMA.	65

Lista de Tablas

TABLA 2.1 RESUMEN DE PROTOCOLOS AXI Y SUS APLICACIONES EN UN CANAL DE VÍDEO.	7
TABLA 3.1 SEÑALES DE RELOJ DEL CANAL DE VÍDEO DE REFERENCIA.	13
TABLA 3.2. DEPENDENCIAS DEL CANAL DE VÍDEO DE REFERENCIA.	14
TABLA 4.1. DEPENDENCIAS DEL SISTEMA DE REFERENCIA Y SUS ADAPTACIONES.	19
TABLA 4.2. PARÁMETROS DE CONFIGURACIÓN DE MICROBLAZE.	21
TABLA 4.3 DOMINIOS DE RELOJ DEL SISTEMA MIGRADO.	25
TABLA 5.1. MODOS DE FUNCIONAMIENTO DEL MÓDULO HLS.	38
TABLA 5.2. COMANDOS DE LA APLICACIÓN DE CONTROL.	44
TABLA 6.1 ANCHO DE BANDA DE MEMORIA MEDIDO EN PROCESADO Y GENERACIÓN DE VÍDEO.	52
TABLA 6.2 UTILIZACIÓN DE RECURSOS LÓGICOS DEL SISTEMA MIGRADO.	53
TABLA 6.3 DESGLOSE DE RECURSOS DE LA INFRAESTRUCTURA DE CONTROL.	54
TABLA 6.4 COSTE DE LA INFRAESTRUCTURA DE INTERCONEXIÓN.	55
TABLA 6.5 COSTE Y RENDIMIENTO DE LA CONFIGURACIÓN DE MICROBLAZE.	56
TABLA 6.6 RECURSOS DEL SISTEMA MIGRADO Y COMPLETO, CON EL INCREMENTO DE LAS AMPLIACIONES.	58
TABLA 6.7 COSTE DE DISTINTAS IMPLEMENTACIONES DE LA ESCALA A GRIS (MÓDULO COMPLETO).	59
TABLA 6.8 ESTIMACIÓN DE CONSUMO DE VIVADO SOBRE LA ARTY S7-50.	60
TABLA 6.9 CONSUMO DE POTENCIA MEDIDO SOBRE LA ARTY S7-50 PARA LOS DISTINTOS SISTEMAS. MARGEN DE ERROR ATRIBUIDO AL 5 % DE TOLERANCIA DE LA RESISTENCIA SHUNT.	61
TABLA 6.10 COMPARATIVA DE POTENCIA ENTRE SISTEMA ORIGINAL Y MIGRADO.	63
TABLA 6.11. ESTIMACIÓN DE CONSUMO DE VIVADO PARA EL SISTEMA DE REFERENCIA Y EL MIGRADO.	64
TABLA 6.12 OCUPACIÓN DE RECURSOS DEL SISTEMA EN AMBAS PLATAFORMAS.	65

TABLA 6.13 RECURSOS DE LÓGICA PROGRAMABLE Y COSTE DE DISPOSITIVOS DISCRETOS Y SoC.	
PRECIOS UNITARIOS DE DIGIKEY SIN IVA [18].	66

Lista de Acrónimos

APM	AXI Performance Monitor
ARM	Advanced RISC Machines
AXI	Advanced eXtensible Interface
BRAM	Block RAM
CMOS	Complementary Metal-Oxide-Semiconductor
DDR	Double Data Rate
DDR3L	Double Data Rate Low Voltage
DSP	Digital Signal Processing
FF	Flip-Flop
FPGA	Field Programmable Gate Array
FPS	Frames Per Second
GPIO	General-Purpose Input/Output
HDL	Hardware Description Language
HDMI	High-Definition Multimedia Interface
HLS	High Level Synthesis
I2C/IIC	Inter-Integrated Circuit
IP	Intellectual Property
LUT	Look Up Table

MIG	Memory Interface Generator
OSD	On-Screen Display
PL	Programmable Logic
PMOD	Peripheral MODule
PS	Processing System
RGB	Red, Green, Blue
RISC	Reduced Instruction Set Computer
SoC	System on a Chip
TMDs	Transition Minimized Differential Signaling
UART	Universal Asynchronous Receiver-Transmitter
VDMA	Video Direct Memory Access
VGA	Video Graphics Array
VTC	Video Timming Controller
XADC	Xilinx Analog-to-Digital Converter

Capítulo 1. Introducción

1.1. Motivación

El procesamiento de vídeo en tiempo real está presente en multitud de aplicaciones, desde la visión artificial hasta los sistemas avanzados de vigilancia o asistencia. Todas comparten requisitos exigentes de ancho de banda y latencia: grandes flujos de píxeles deben capturarse, procesarse y transmitirse con un retardo reducido y predecible.

Las FPGA (*Field Programmable Gate Array*) resultan especialmente adecuadas para esta tarea, pues procesan los píxeles mediante circuitos hardware dedicados que garantizan un alto rendimiento y una latencia determinista.

Ahora bien, un canal de vídeo completo no se limita al procesamiento de datos. También requiere una unidad de control que inicialice sensores, configure los periféricos y coordine el sistema, tareas que se resuelven con naturalidad mediante software. La solución habitual es emplear un SoC (*System on a Chip*) basado en FPGA, que combina lógica programable con un procesador capaz de ejecutar código de alto nivel e incorpora interfaces predefinidas que simplifican el desarrollo. Esa integración, sin embargo, encarece la plataforma y ata el sistema a unos recursos fijos que no toda aplicación aprovecha.

De esta cuestión surge la motivación principal de este proyecto: explorar el potencial de una plataforma de bajo coste basada en una FPGA discreta para albergar un canal de vídeo en tiempo real completo, reproduciendo sobre la lógica programable las funciones que en un SoC vienen integradas. El interés es doble: ajustar el diseño a las necesidades reales del sistema y evaluar el coste de prescindir del procesador

dedicado, abriendo la puerta a plataformas más asequibles y flexibles para la misma tarea.

1.2. Objetivos específicos

El objetivo principal de este trabajo es migrar un canal de vídeo en tiempo real, desarrollado originalmente sobre una plataforma FPGA SoC, a una placa de bajo coste Arty S7-50 basada en una FPGA discreta, ampliarlo con nuevos subsistemas, y caracterizar el coste de dicha migración. De este eje central se derivan los siguientes objetivos específicos:

- Reproducir sobre la FPGA discreta las funciones que el proyecto de referencia delegaba en su sistema integrado, conservando el comportamiento del canal de vídeo.
- Ampliar el sistema resultante con nuevas funciones, comprobando que la arquitectura migrada tiene margen de mejora y puede asumir tareas más complejas.
- Validar y caracterizar el sistema con medidas reales, comparándolo con la plataforma de referencia en recursos, rendimiento y consumo.

1.3. Metodología

El desarrollo del proyecto ha seguido un proceso incremental. En primer lugar, se analizó el canal de vídeo de referencia implementado sobre una FPGA SoC, se identificaron sus dependencias y se estudiaron las posibles adaptaciones.

A continuación, el proyecto se migró a una FPGA discreta reproduciendo las dependencias de forma progresiva, para validar cada subsistema por separado. Se incorporó un procesador *soft-core*, un controlador de memoria externa basado en lógica programable y los periféricos necesarios, todo ello integrado en una arquitectura de interconexión ajustada. El desarrollo se lleva a cabo con las herramientas de AMD/Xilinx: Vivado y su entorno *IP (Intellectual Property) Integrator* para la integración de los bloques IP, y Vitis HLS (*High-Level Synthesis*) para el diseño de alto nivel.

El sistema se valida funcionalmente sobre el montaje físico y se caracteriza con medidas reales, apoyándose en un subsistema de monitorización desarrollado a tal efecto. Las decisiones de diseño se contrastan con sus alternativas para justificar su elección.

1.4. Organización de la memoria

La memoria se estructura en siete capítulos. Tras esta introducción, el Capítulo 2 presenta los fundamentos del procesado de vídeo sobre FPGA y las tecnologías empleadas en este proyecto. El Capítulo 3 describe el sistema de referencia del que parte la migración. El Capítulo 4 detalla la migración del canal de vídeo a la FPGA discreta, conformando el núcleo técnico del trabajo. En el Capítulo 5 se amplía el sistema con nuevas funcionalidades de monitorización y un módulo de procesamiento de vídeo. El Capítulo 6 recoge la validación y caracterización del resultado, comparando con el sistema de referencia y contrastando los resultados obtenidos. Por último, el Capítulo 7 expone las conclusiones y plantea futuras líneas de trabajo.

Capítulo 2. Estado del arte

Este capítulo presenta los fundamentos y las tecnologías sobre los que se asienta el trabajo, centrándose en los necesarios para seguir el desarrollo del proyecto. Se introduce cómo se implementan las funcionalidades básicas de una cadena de vídeo en un circuito hardware, para introducir después los elementos que lo hacen posible sobre una FPGA.

2.1. Procesado de vídeo sobre FPGA

Un sistema de vídeo en tiempo real captura una secuencia de imágenes, la procesa y la presenta en una salida, manteniendo un flujo continuo de píxeles. Para lograr ese transporte constante de datos, los diseños en FPGA suelen emplear un protocolo basado en *stream*, de manera que la comunicación entre bloques es continua y coherente. Las distintas codificaciones de los sensores, como RGB (*Red, Green, Blue*), se transforman internamente a este protocolo de flujo.

Aunque muchas operaciones de vídeo se realizan directamente sobre el flujo, otras requieren almacenar imágenes completas. Por ejemplo, cuando la captura y la visualización operan a tasas distintas. Para ello se emplean los *frame buffers*, regiones de memoria donde se almacenan los fotogramas capturados hasta su lectura para la salida. A fin de evitar el acceso simultáneo de escritura y lectura sobre un mismo fotograma, se recurre a esquemas de doble o triple *buffer* con un mecanismo de arbitraje, como ilustra la Figura 2.1.

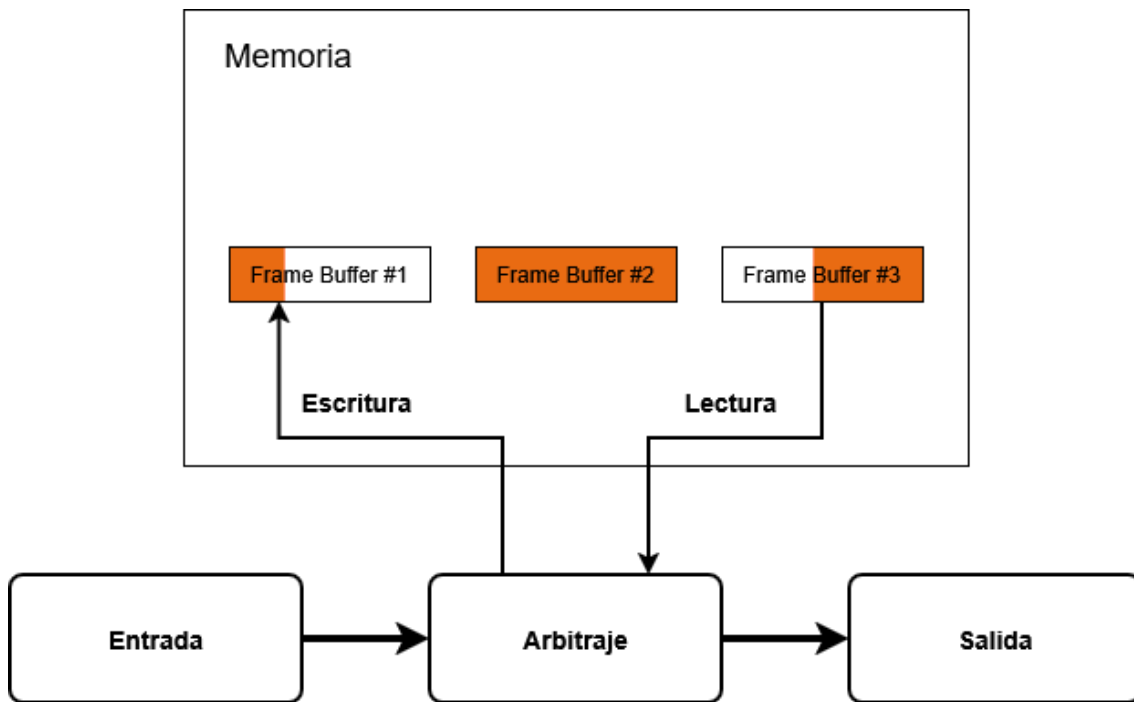


Figura 2.1 Ejemplo de aplicación de frame buffers arbitrados.

La capacidad necesaria suele ser elevada. Un fotograma a 1080p (1920×1080) y 24 bits por píxel ocupa 6,22 MB, de modo que, según las prestaciones de la Serie 7 de AMD/Xilinx [1], muchos dispositivos de gama media no disponen de esa memoria en todo el chip. Por ello, los sistemas de vídeo recurren a memoria externa DDR (*Double Data Rate*), que aporta la capacidad y el ancho de banda necesarios para la transferencia continua de fotogramas.

El procesamiento del flujo continuo se implementa mediante segmentación o *pipelining*. Como se define en [2], el algoritmo se divide en etapas secuenciales sincronizadas por un reloj común, de manera que, una vez lleno el *pipeline*, cada etapa procesa un píxel distinto en el mismo ciclo, según ilustra el ejemplo de la Figura 2.2. A cambio de una latencia inicial de tantos ciclos como etapas, esta técnica eleva notablemente el rendimiento al procesar varios píxeles en paralelo, lo que hace de las FPGA una plataforma idónea para el vídeo en tiempo real.

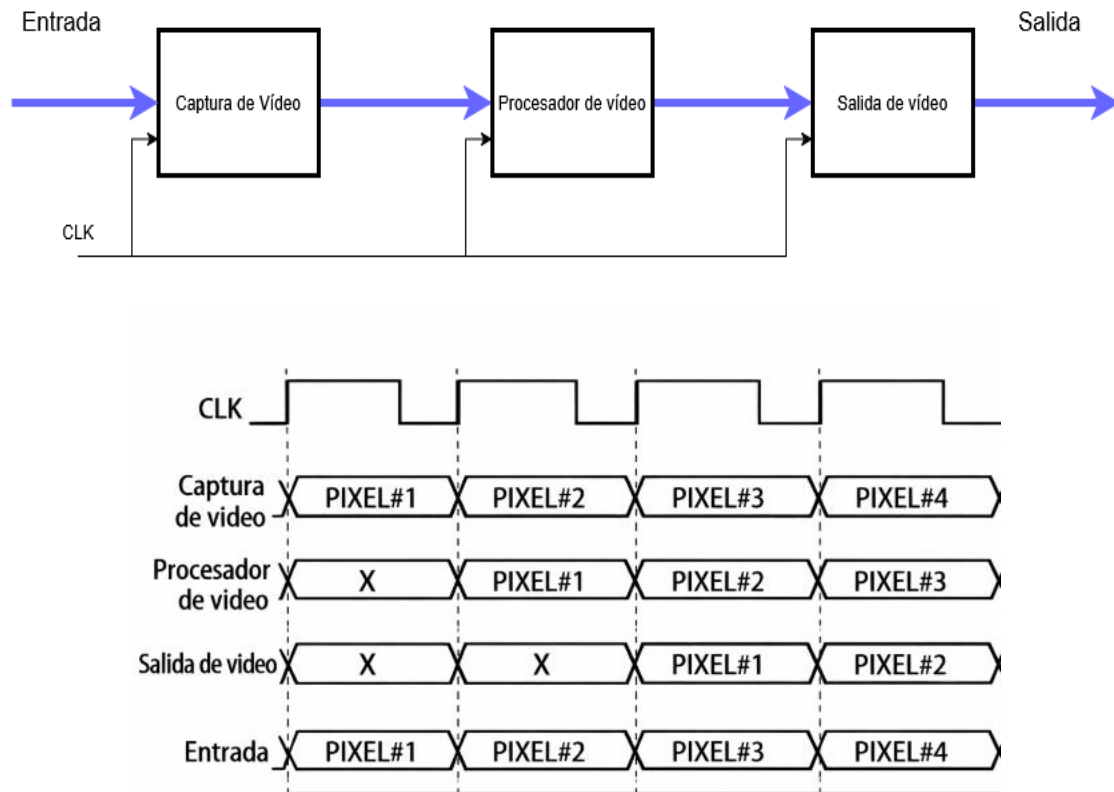


Figura 2.2 Ejemplo de pipeline aplicado a cadena de vídeo.

2.2. Protocolos AXI

La interconexión de los distintos bloques que componen el sistema se realiza mediante el conjunto de protocolos AXI (*Advanced eXtensible Interface*), un estándar ampliamente adoptado en los diseños sobre FPGA de AMD/Xilinx. Su interfaz permite que dispositivos maestros y esclavos realicen transacciones independientes de escritura y lectura mediante una serie de canales paralelos, como se define en [3].

La Tabla 2.1 resume la función principal de cada protocolo AXI.

Protocolo	Uso característico
AXI4-Lite	Acceso a registros de control y configuración de periféricos
AXI4	Transferencias a memoria de gran volumen, mediante ráfagas
AXI4-Stream	Transmisión de un flujo continuo de datos sin direccionamiento

Tabla 2.1 Resumen de protocolos AXI y sus aplicaciones en un canal de vídeo.

La distinción es importante para identificar dónde se emplea cada variante en el sistema. AXI4-Stream se emplea para transmitir el flujo de píxeles entre bloques de procesado, AXI4 para el acceso a los *frame buffers* en memoria externa, y AXI4-Lite para que el procesador configure los periféricos.

2.3. Plataformas FPGA SoC y FPGA discretas

Las FPGA pueden clasificarse en dos modalidades relevantes para este trabajo. Las plataformas SoC integran en un mismo chip Lógica Programable (*Programmable Logic, PL*) y un Sistema de Procesamiento (*Processing System, PS*), que incorpora procesadores dedicados, periféricos, controladores de memoria y buses, resueltos en silicio dedicado.

En contraste, las FPGA discretas están formadas esencialmente por lógica programable. Las funcionalidades de procesamiento, control y acceso a memorias externas deben sintetizarse sobre los propios recursos lógicos.

Esta división funcional entre ambos dispositivos condiciona el área de aplicación de cada una. Como se discute en [4], las FPGA SoC suelen destinarse a aplicaciones donde el rendimiento de sus procesadores cobra valor, mientras que las FPGA discretas tienen contextos de aplicación más orientados a la simplicidad arquitectónica o la optimización de costes.

En este trabajo se cuantifica la realidad de esta separación. Al migrar un sistema concebido en un SoC a una FPGA discreta, las funciones delegadas en el PS se tendrán que reproducir mediante lógica programable, lo que permitirá caracterizar los costes de la migración y el rendimiento que se obtiene.

2.4. Procesadores soft-core y MicroBlaze

Para suplir la ausencia de un procesador físico en la FPGA discreta se puede emplear un procesador *soft-core*: una unidad de procesamiento descrita mediante HDL (*Hardware Description Language*) y, por tanto, sintetizable sobre la lógica programable.

MicroBlaze es el procesador *soft-core* desarrollado por AMD/Xilinx para sus dispositivos FPGA. Está basado en una arquitectura RISC (*Reduced Instruction Set Computer*) de 32 bits y es altamente configurable. Según su guía oficial en [5], su tamaño, sus cachés y sus unidades de cálculo pueden ajustarse al nivel que la aplicación lo requiera.

Como se indica en el estudio de [6], la decisión entre emplear un procesador físico o un *soft-core* se fundamenta en los requisitos de área ocupada, consumo de potencia, rendimiento o velocidad que el sistema final demanda. Siguiendo esta línea, en este proyecto se valorarán los costes y beneficios de implementar la unidad de control mediante un *soft-core*.

2.5. Síntesis de Alto Nivel (HLS)

La Síntesis de Alto Nivel (*High-Level Synthesis*, HLS) es una metodología de diseño que permite describir circuitos hardware mediante lenguajes de alto nivel como C++, y generar automáticamente una implementación sintetizable sobre una FPGA. Frente al desarrollo basado en HDL, este enfoque facilita la implementación de algoritmos complejos.

Aunque el comportamiento funcional se describa mediante código de alto nivel, el hardware generado debe cumplir los mismos requisitos de temporización, rendimiento y utilización de recursos que una implementación

convencional en HDL. Para ello, las herramientas de HLS permiten definir objetivos de síntesis y aplicar directivas de optimización que controlan aspectos como el *pipelining* o el paralelismo del circuito generado.

En este trabajo, HLS se emplea para diseñar un módulo de procesamiento de vídeo en tiempo real.

Capítulo 3. Canal de vídeo de referencia basado en FPGA SoC

3.1. Descripción general del sistema de referencia

El punto de partida de este proyecto consiste en el estudio de un canal de vídeo en tiempo real desarrollado por el Departamento de Tecnología Electrónica de la UMA (Universidad de Málaga) sobre una placa Zybo Z7, basada en un SoC de la familia Zynq-7000.

El sistema captura imágenes desde una cámara CMOS (*Complementary Metal-Oxide-Semiconductor*) MT9V111 de Aptina, las almacena temporalmente en la memoria DDR3 de la plataforma y, finalmente, genera la señal de vídeo HDMI (*High-Definition Multimedia Interface*) en resolución VGA (*Video Graphics Array*). La cámara se conecta por dos puertos PMOD (Peripheral MODule), mientras que la salida de vídeo utiliza el conector HDMI integrado en la plataforma.

Siguiendo la arquitectura habitual de los dispositivos Zynq descrita en [7], el diseño se reparte entre la lógica programable, que contiene los bloques encargados del procesamiento de vídeo, y el Sistema de Procesamiento, responsable de las tareas de control y del acceso a recursos externos. La Figura 3.1 presenta el diagrama de bloques del sistema.

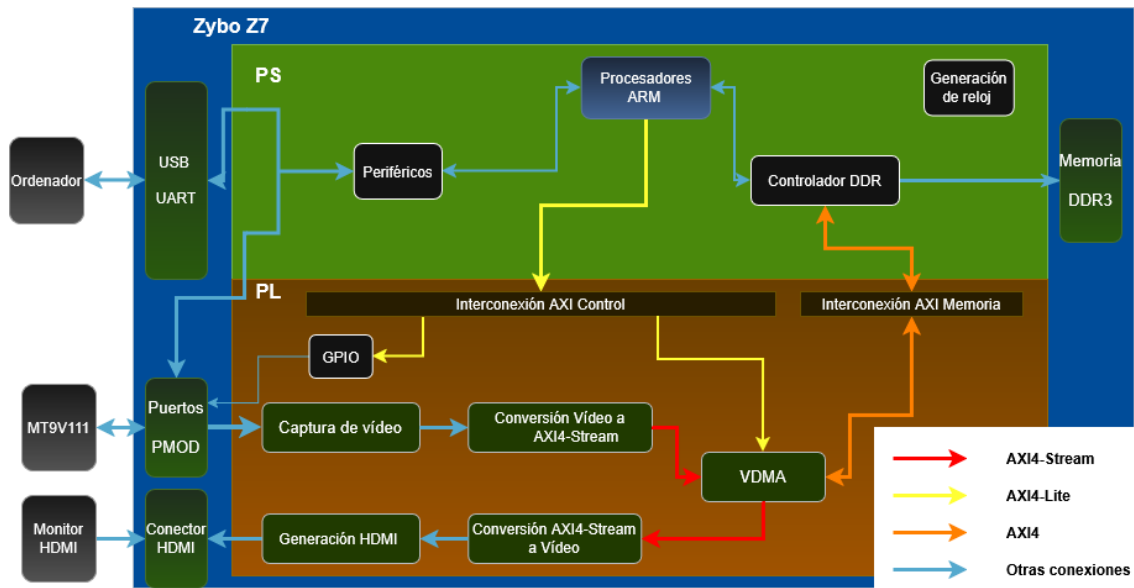


Figura 3.1 Arquitectura general del sistema de referencia.

El objetivo de este capítulo es comprender el funcionamiento de la arquitectura de referencia e identificar las dependencias asociadas al sistema de procesamiento, las cuales deberán sustituirse durante la migración a una FPGA discreta.

3.2. Cadena de procesamiento de vídeo (PL)

El funcionamiento del canal de vídeo se puede describir siguiendo el camino recorrido por un fotograma desde su captura hasta su visualización. La Figura 3.2 representa el flujo de vídeo completo, el cual se estudiará de izquierda a derecha.

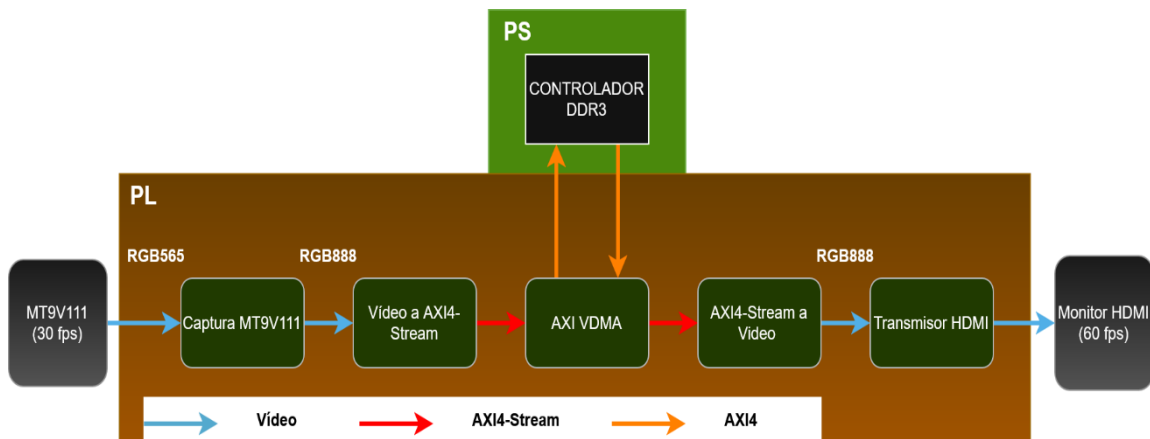


Figura 3.2 Cadena de procesamiento de vídeo del sistema de referencia.

El sensor MT9V111 entrega imágenes a 30 fps (*Frames Per Second*) que son capturadas por un bloque RTL (*Register-Transfer Level*) específico, incluido en el diseño original. Este módulo adapta la interfaz física de la cámara y reconstruye los píxeles al formato RGB888, siendo así legibles por el bloque de conversión al protocolo AXI4-Stream.

El flujo generado se conecta a un bloque AXI VDMA (*Video Direct Memory Access*) [8], que traduce el flujo AXI4-Stream a transacciones AXI4 hacia la memoria DDR3 externa, que actúa como soporte para implementar un triple *frame buffer*.

Además, el modo *Genlock* del VDMA arbitra el acceso a los distintos *buffers*, permitiendo que la captura a 30 fps y la visualización a 60 fps coexistan sin que ocurran accesos simultáneos a un mismo fotograma.

Posteriormente, el canal de lectura del VDMA recupera los fotogramas desde la memoria y los reintroduce en el canal de vídeo como un nuevo flujo AXI4-Stream.

A continuación, se reconstruye la interfaz de vídeo RGB888 a partir del flujo recibido y se generan las señales de sincronismo asociadas a la resolución VGA empleando un bloque *Video Timing Controller* (VTC).

Finalmente, un bloque transmisor HDMI genera la señal enviada al monitor, completando el camino del canal de vídeo en tiempo real.

En paralelo a esto, se emplea un bloque *Clocking Wizard* para generar las señales de reloj necesarias para captura y salida de vídeo. El reloj de referencia del sistema que coordina el resto de los periféricos se genera desde el PS. La Tabla 3.1 recoge las frecuencias, fuentes y aplicaciones de estos relojes.

Frecuencia	Aplicación	Origen
25 MHz	Reloj de píxel (HDMI)	Oscilador (Clocking Wizard)
125 MHz	Reloj serie de datos (HDMI)	Oscilador (Clocking Wizard)
24 MHz	Reloj maestro de la cámara	Oscilador (Clocking Wizard)
150 MHz	Sincronización de interfaces AXI	PS

Tabla 3.1 Señales de reloj del canal de vídeo de referencia.

3.3. Funciones del Sistema de Procesamiento

A pesar de que el procesamiento de vídeo reside íntegramente en la lógica programable, su funcionamiento depende de diversos servicios proporcionados por el PS.

Principalmente, los procesadores ARM (*Advanced RISC Machines*) ejecutan la aplicación software encargada de inicializar el sistema. Entre sus funciones, se encuentran la configuración de la cámara mediante una interfaz I2C (*Inter-Integrated Circuit*), la inicialización del bloque VDMA y la asignación de las regiones de los *frame buffers* en memoria. La Figura 3.3 resume la secuencia de inicialización del canal de vídeo.

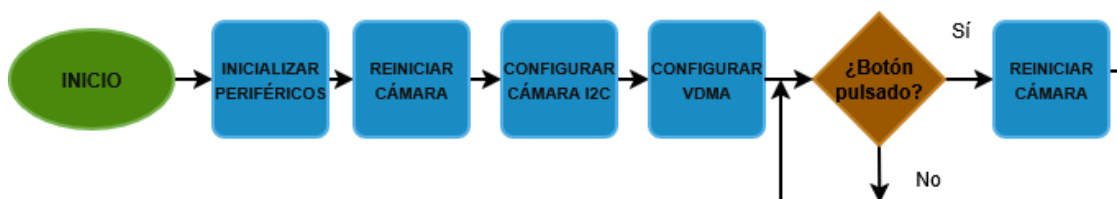


Figura 3.3 Flujograma de inicialización del proyecto de referencia.

Además, el PS integra una interfaz UART (*Universal Asynchronous Receiver-Transmitter*) utilizada para transmitir mensajes de depuración a un terminal externo, así como el controlador DDR3 necesario para acceder a la memoria. Del mismo modo, el PS genera una señal de reloj sincronizada con la memoria externa y los procesadores, que gobierna todas las interfaces AXI.

3.4. Consideraciones para la migración del sistema

El análisis del sistema de referencia revela una división con consecuencias directas para la migración. Por un lado, la cadena de procesamiento de vídeo reside íntegramente en la lógica programable y se construye sobre bloques IP compatibles con toda la Serie 7.

Por otro, se han identificado las distintas funcionalidades del sistema que se resuelven mediante elementos integrados en el PS o específicos de placa de desarrollo Zybo Z7. La Tabla 3.2 recoge las dependencias detectadas durante este estudio.

Funcionalidad	Implementación en Zybo Z7
Ejecución de la aplicación software	Procesadores ARM Cortex-A9 (PS)
Configuración de la cámara	Interfaz I2C integrada (PS)
Mensajes de depuración	UART integrada (PS)
Acceso a memoria externa	Controlador DDR integrado (PS)
Sincronización del sistema	Infraestructura de reloj (PS)
Salida de vídeo	Conector HDMI de la placa

Tabla 3.2. Dependencias del canal de vídeo de referencia.

La migración a una FPGA discreta, por tanto, no consiste en rediseñar el canal de vídeo, sino en sustituir estas dependencias por alternativas basadas en lógica programable o hardware externo.

El capítulo siguiente describe las soluciones adoptadas para reproducir dichas funciones en una plataforma Arty S7-50.

Capítulo 4. Migración del sistema a una FPGA discreta

En este capítulo se presenta el sistema propuesto sobre la nueva plataforma, justificando las decisiones de diseño tomadas para sustituir cada una de las dependencias identificadas en el proyecto de referencia. En el apartado final, se valida el funcionamiento del canal de vídeo migrado.

4.1. Plataforma objetivo: Arty S7-50

La plataforma objetivo es la placa de desarrollo Arty S7-50 de Digilent, basada en una FPGA Spartan-7 XC7S50 de AMD/Xilinx. Su elección responde directamente a la motivación del proyecto: demostrar que un canal de vídeo completo puede implementarse sobre un dispositivo de bajo coste, sin periféricos integrados ni procesadores dedicados.

La migración planteaba dos retos distintos: la sustitución del Sistema de Procesamiento por una infraestructura sintetizada sobre lógica programable, y la salida de vídeo a través de un puerto PMOD genérico, al carecer la placa Arty S7 de conector HDMI. Para abordarlos por separado, el sistema se implementó en primer lugar sobre una plataforma Nexys Video, cuyo conector HDMI integrado permitió validar el funcionamiento de la arquitectura propuesta. Posteriormente, el desarrollo se trasladó a la Arty S7-50, quedando como último detalle la adaptación física de la salida, que se aborda en el apartado 4.5.

La plataforma incorpora los recursos necesarios para implementar el sistema propuesto, entre los que destacan una memoria DDR3L de 256 MB,

varios conectores PMOD de expansión y una interfaz USB-UART para programación y comunicación con usuario, según su manual de referencia [9].

La Figura 4.1 muestra la plataforma Arty S7-50 y los componentes involucrados en el proyecto.

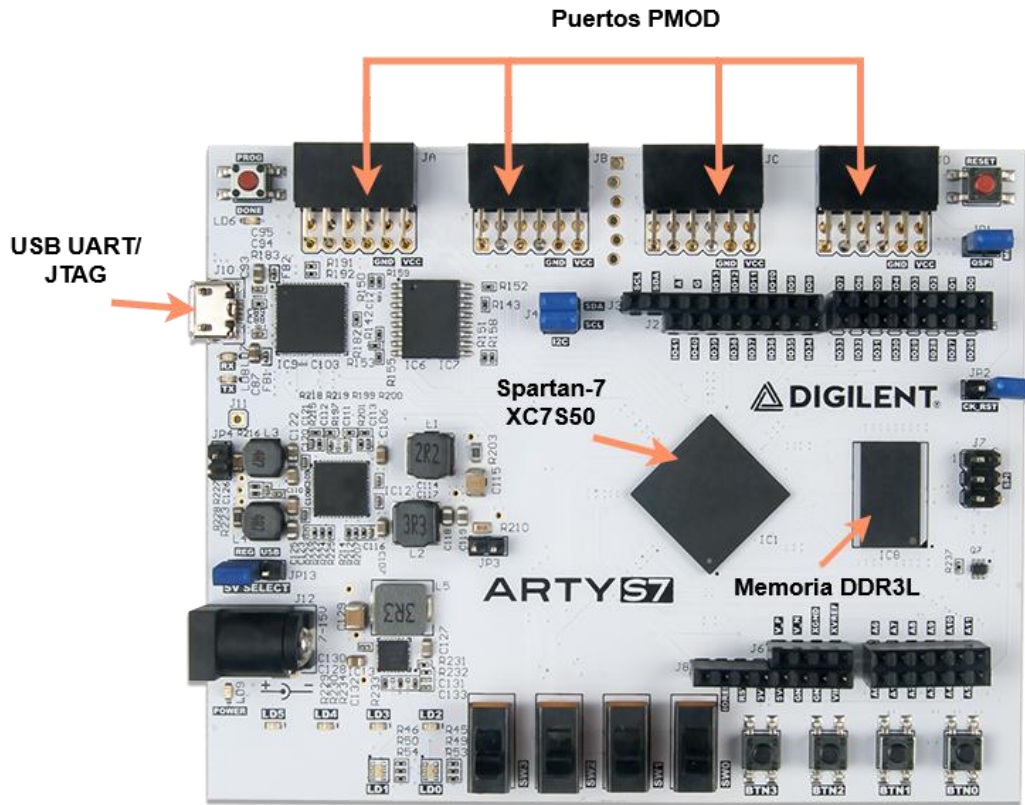


Figura 4.1. Plataforma Arty S7-50 y componentes empleados en el proyecto. [9]

4.2. Arquitectura general del sistema migrado

La Figura 4.2 muestra el diagrama del sistema propuesto sobre la placa Arty S7-50.

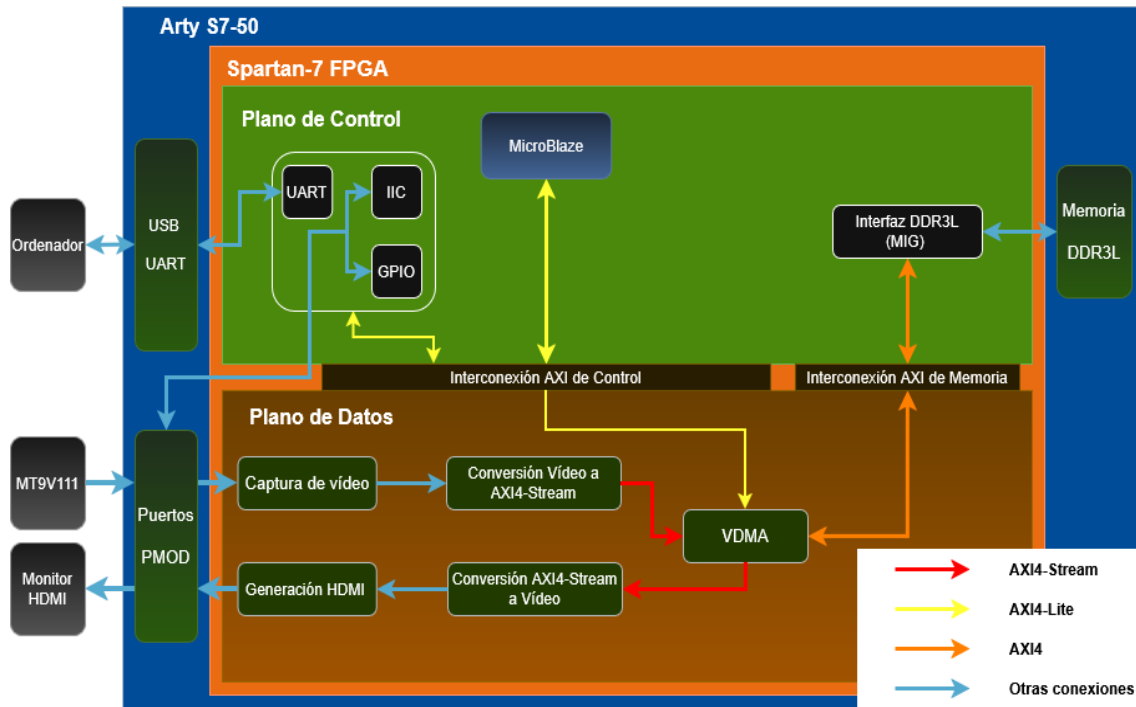


Figura 4.2. Arquitectura del sistema de vídeo migrado a la Arty S7-50.

Con fines descriptivos y de diseño, el sistema se ha organizado en dos planos. El Plano de Datos contiene la cadena de procesamiento de fotogramas encargada de la captura, almacenamiento y generación de la salida de vídeo, manteniendo la estructura del proyecto original salvo adaptaciones en la sincronización de dominios de reloj.

Por su parte, el Plano de Control engloba los componentes que sustituyen las funcionalidades del Sistema de Procesamiento, constituyendo la aportación principal de este capítulo. La interconexión entre ambos planos se realiza por estructuras AXI, que serán adaptadas a los requisitos de cada dominio.

La Tabla 4.1 resume las adaptaciones realizadas respecto a las dependencias recogidas en la Tabla 3.2, y serán descritas con detalle en los apartados siguientes.

Funcionalidad en Zybo Z7	Solución en Arty S7-50
Procesadores ARM Cortex-A9	Procesador <i>soft-core</i> MicroBlaze
I2C Integrado	Bloque IP AXI IIC
UART integrada	Bloque IP AXI UART Lite
Controlador DDR3	Bloque IP MIG DDR3L
Infraestructuras de reloj	Dominios de reloj independientes por funcionalidad
Conector HDMI de la placa	Salida diferencial TMDS por puerto PMOD

Tabla 4.1. Dependencias del sistema de referencia y sus adaptaciones.

4.3. Plano de Control

4.3.1. Sistema de control basado en MicroBlaze

La diferencia fundamental entre la plataforma de referencia y una FPGA discreta es la ausencia de un procesador *hard-core*. Resulta necesario, por tanto, incorporar un mecanismo capaz de inicializar y configurar el canal de vídeo y que sea, al mismo tiempo, compatible con el resto de la arquitectura.

Para resolver esta necesidad se ha optado por el procesador *soft-core* MicroBlaze por tres razones principales. En primer lugar, se integra por defecto en las herramientas de AMD/Xilinx, evitando los costes de emplear núcleos de terceros. En segundo lugar, dispone de una interfaz AXI4-Lite maestra, lo que le permite gobernar los mismos periféricos AXI que en el sistema original controlaba el PS. Por último, y como consecuencia de lo anterior, permite reutilizar con modificaciones mínimas la aplicación software del sistema de referencia, manteniendo así el paradigma de control original.

Una vez seleccionada esta arquitectura de control, su configuración se ha dimensionado al perfil real de la aplicación. Dado que la participación de MicroBlaze se limita a tareas puntuales de inicialización y configuración, se adoptó una implementación *bare-metal* orientada a minimizar la ocupación de recursos: una memoria local de 64 kB en BRAM (*Block RAM*), suficiente para alojar la aplicación completa con acceso a instrucciones y datos en latencia mínima, lo que hace innecesario el acceso a la DDR3L durante la ejecución y permite prescindir de las cachés. La frecuencia se fijó en 100 MHz, holgada para las reducidas exigencias computacionales, con la optimización orientada a rendimiento habilitada. La Tabla 4.2 resume la configuración resultante.

La Figura 4.3 muestra la implementación resultante, a la que únicamente se añade el módulo de depuración básico necesario para ejecutar la aplicación software, según se especifica en la guía de MicroBlaze [5].



Figura 4.3. Implementación del procesador MicroBlaze.

Parámetro	Configuración
Memoria	64 kB (BRAM)
Memoria caché	Desactivada
Gestor de interrupciones	Desactivado
Operadores matemáticos hardware	Desactivados
Interfaz AXI	Habilitada para datos
Frecuencia de funcionamiento	100 MHz

Módulo de depuración	Básico
Optimización de implementación	Rendimiento

Tabla 4.2. Parámetros de configuración de MicroBlaze.

4.3.2. Interfaz de memoria DDR3L (MIG)

Debido a que se mantiene la cámara MT9V111 y la salida HDMI a 60 Hz, el sistema migrado debe disponer también de una arquitectura de *frame buffers*. Se consideró emplear la BRAM del dispositivo para implementar los *buffers*, pero resultó ser insuficiente. Según la ficha técnica de la Spartan-7 [1], el chip XC7S50 dispone de 337,5 KB de BRAM, mientras que un único fotograma en resolución VGA necesita:

$$(640 \cdot 480) \frac{\text{píxeles}}{\text{fotograma}} \cdot 3 \frac{\text{bytes}}{\text{píxel}} = 921,6 \text{ KB/fotograma}$$

La capacidad necesaria es casi el triple, sin contar que se debe implementar más de un *frame buffer*. Por ello, se optó por hacer uso de la memoria externa presente en la plataforma Arty S7-50 para albergar un triple *buffer*.

El acceso a la memoria DDR3L se resuelve mediante el bloque IP *Memory Interface Generator* (MIG)[10] de AMD/Xilinx, que genera sobre la lógica programable el controlador físico de la memoria externa y expone una interfaz AXI4 hacia el resto del sistema. Su configuración se ha realizado siguiendo las especificaciones recomendadas en [9], salvo dos parámetros ajustados a las necesidades del proyecto: la frecuencia de operación y el ancho de la interfaz AXI4.

Como se indica en [9], la memoria DDR3L admite una frecuencia física de hasta 333,5 MHz, pero en lugar de maximizarla, el criterio seguido fue ajustarla a la demanda real del sistema. Para dimensionar correctamente la frecuencia,

se comparó el ancho de banda que la memoria proporciona con el requerido por el sistema.

Con un bus de 16 bits de la memoria DDR3L y doble tasa de transferencia, una frecuencia de 325 MHz proporciona un ancho de banda máximo teórico de:

$$2 \text{ Bytes} \cdot 2 \cdot 325 \text{ MHz} = 1300 \text{ MB/s}$$

Mientras que las transferencias continuas de vídeo VGA en formato RGB888 a 60 y a 30 fps consumen, respectivamente:

$$(640 \cdot 480) \frac{\text{píxeles}}{\text{fotograma}} \cdot 60 \frac{\text{fotogramas}}{\text{s}} \cdot 3 \frac{\text{bytes}}{\text{píxel}} = 55,3 \text{ MB/s}$$

$$(640 \cdot 480) \frac{\text{píxeles}}{\text{fotograma}} \cdot 30 \frac{\text{fotogramas}}{\text{s}} \cdot 3 \frac{\text{bytes}}{\text{píxel}} = 27,65 \text{ MB/s}$$

Incluso sumando los canales de lectura y escritura del VDMA, la demanda apenas alcanza una fracción de la capacidad disponible. En consecuencia, operar a la frecuencia máxima de la memoria no aporta ventaja práctica alguna, por lo que se adopta la configuración de ~325 MHz.

El bloque MIG genera además una señal de reloj *ui_clk* (*User Interface Clock*), que sincroniza las transacciones realizadas con su interfaz AXI4 y se deriva del reloj físico de la memoria mediante una relación 4:1, resultando en ~81,25 MHz. Esta relación implica que cada ciclo de la interfaz AXI4 equivale a una transferencia efectiva de 128 bits en la memoria, por lo que se configuran las interfaces AXI4 de los bloques VDMA y MIG con ese ancho para aprovechar al máximo el ancho de banda disponible en las transferencias de fotogramas.

Dado que las transferencias entre el bloque VDMA y la memoria deben estar alineadas por *ui_clk*, se ha adoptado también esta señal como el reloj principal del Plano de Datos. Esta decisión sincroniza la cadena de vídeo y el subsistema de memoria, simplificando el diseño y garantizando la máxima coherencia en el transporte de fotogramas.

4.3.3. Periféricos de comunicación

Las funciones de configuración y supervisión del sistema de referencia se mantienen mediante tres bloques IP conectados a la red de control.

La configuración de la cámara MT9V111 se realiza mediante un bloque AXI IIC, que proporciona la interfaz I2C necesaria para acceder a los registros internos del sensor. Adicionalmente, se incluye un núcleo AXI GPIO (*General-Purpose Input/Output*) para controlar la señal de reinicio de la cámara desde software.

La comunicación serie con el ordenador se establece a través de un bloque AXI UART Lite, conectado al conversor USB-UART de la placa. Se ha elegido esta variante frente al núcleo AXI UART16550 completo debido a que únicamente se requiere transmitir mensajes de depuración y recibir comandos simples, sin necesidad de funcionalidades adicionales.

4.4. Infraestructuras de interconexión y sincronización

La separación en planos funcionales se extiende a las infraestructuras de interconexión y a la distribución de reloj, permitiendo dimensionar cada una según sus requisitos.

La red de control se implementa mediante un bloque AXI InterConnect, donde MicroBlaze actúa como único maestro AXI4-Lite, mientras que los distintos periféricos y registros configurables (UART, IIC, GPIO y VDMA) se conectan como esclavos. Dado que estas comunicaciones se limitan a lecturas y escrituras de registros, los requisitos de ancho de banda son reducidos.

Las transferencias de vídeo con la memoria, en cambio, se canalizan a través de un bloque AXI SmartConnect, que conecta los canales del VDMA con el MIG y está optimizado para las transferencias de alto rendimiento que exige el flujo continuo de fotogramas, como se especifica en [11].

El uso de redes independientes aísla el tráfico de control del de datos y reduce la lógica de interconexión, al no mezclar protocolos AXI distintos en una misma red. La Figura 4.4 muestra la organización completa de las interconexiones y sus dispositivos asociados.

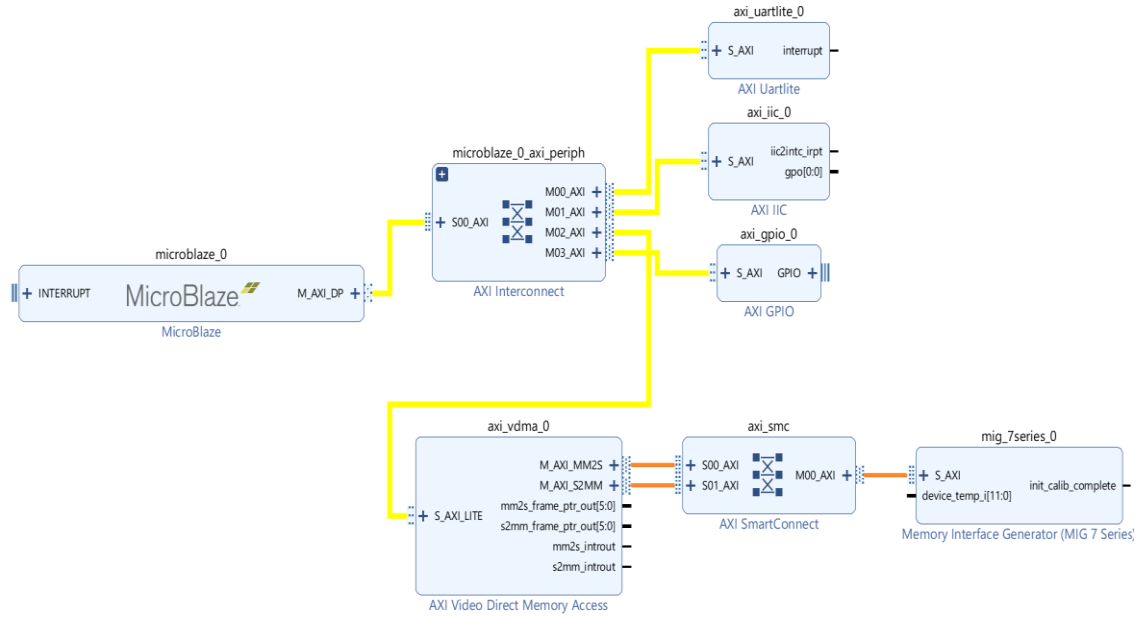


Figura 4.4 Redes de interconexión de control (color amarillo) y de memoria (color naranja).

La distribución de relojes sigue la misma separación. En ausencia del PS, todas las señales de sincronización deben generarse en la propia lógica programable a partir del oscilador de la placa. Un bloque de gestión de reloj produce las frecuencias asociadas al Plano de Control, el controlador MIG deriva el reloj del dominio de datos, y los relojes de píxel y serialización HDMI se generan igualmente con el bloque de gestión de reloj, como en el sistema de referencia.

De este modo, el sistema queda organizado en tres dominios principales: 100 MHz para MicroBlaze, 81,25 MHz (*ui_clk*) para el Plano de Datos y 50 MHz para la infraestructura AXI4-Lite y las interfaces de control de los periféricos.

La frecuencia de 50 MHz se selecciona por dos motivos. En primer lugar, garantiza la compatibilidad con los bloques IP empleados, algunos de los cuales imponen restricciones sobre la relación entre los relojes de control y de datos. En segundo lugar, mantiene una relación síncrona 2:1 con los 100 MHz de MicroBlaze, reduciendo la lógica de adaptación necesaria entre ambos dominios en la infraestructura AXI, tal y como se indica en la documentación del AXI Interconnect [12].

La Tabla 4.3 resume los dominios de reloj que se han asignado.

Dominio	Función	Origen
100 MHz	Operación de MicroBlaze	Clocking Wizard (Oscilador)
81,25 MHz	Sincronización Plano de Datos: Interfaces AXI4 y AXI4-Stream	Interno del MIG
50 MHz	Sincronización Plano de Control: Interfaces AXI4-Lite	Clocking Wizard (Oscilador)
25 MHz	Reloj de píxel de la salida HDMI	Clocking Wizard (Oscilador)
125 MHz	Reloj de serialización TMDS	Clocking Wizard (Oscilador)
23,81 MHz	Reloj de cámara	Clocking Wizard (Oscilador)

Tabla 4.3 Dominios de reloj del sistema migrado.

Conviene señalar que las frecuencias generadas para la cámara y la salida HDMI no coinciden exactamente con las nominales de estos elementos, al derivarse del oscilador disponible en la placa mediante división entera. El sensor opera a 23,81 MHz frente a los 24 MHz nominales, y la salida de vídeo a 25 MHz frente a los 25,175 MHz que corresponden a la resolución VGA. Esta pequeña desviación no impide el funcionamiento del canal, pero se traduce en una ligera reducción de la tasa de fotogramas efectiva respecto a la máxima, cuantificada en el apartado 6.2

4.5. Adaptación de salida HDMI

Dado que la plataforma Arty S7-50 carece de un conector HDMI, fue necesario desarrollar una solución para mantener la capacidad de visualización del sistema.

La salida HDMI transmite el vídeo mediante señalización TMDS (*Transition Minimized Differential Signaling*), que codifica cada canal de color sobre un par diferencial. Como se ve en [1], los bancos de E/S de los dispositivos Spartan-7 soportan de forma nativa este estándar eléctrico. Por tanto, la ausencia de un conector no impide generar la señal de salida, aunque sí se requiere una conexión física adecuada hacia el exterior.

De acuerdo con el manual de referencia de la Arty S7-50 [9], los pines del puerto JB se organizan en pares diferenciales adyacentes, compatibles con las señales generadas por el bloque IP transmisor HDMI. Por tanto, se elige dicho puerto como punto de conexión para la salida de vídeo.

Para establecer la conexión con el monitor se evaluó inicialmente usar un módulo de expansión PMOD HDMI comercial. No obstante, la disposición de las señales diferenciales en este periférico se organizaba verticalmente, resultando incompatible con la distribución horizontal de los pares en el puerto JB.

Como alternativa, se optó por adaptar un cable HDMI comercial para conectarlo directamente a los pines del puerto. Con ayuda del esquema de la Figura 4.5, se identificaron los conductores correspondientes a los tres canales de datos TMDS, el canal de reloj y las líneas de alimentación, soldándolos posteriormente a un terminal de pines compatible con el puerto.

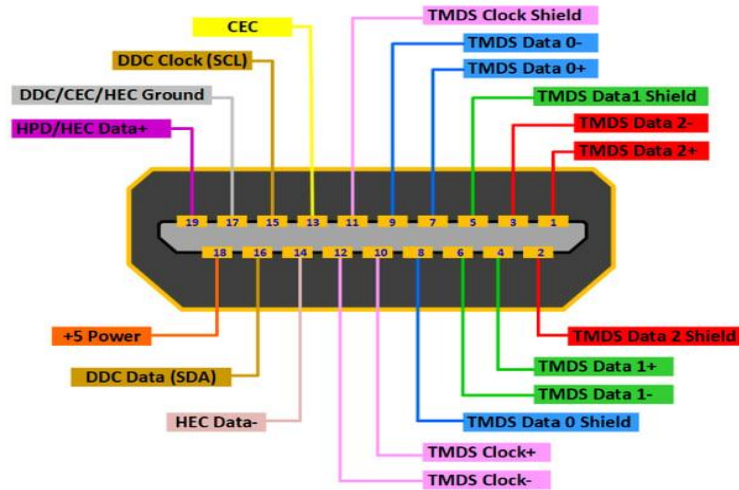


Figura 4.5. Descripción de pines de un conector HDMI (vista trasera).[13]

Dado que el objetivo de esta adaptación era validar la transmisión de vídeo, únicamente se conectaron las señales imprescindibles. Las líneas auxiliares del estándar HDMI que intervienen en la negociación con el monitor no resultaron necesarias.

La Figura 4.6 muestra el esquema de conexión resultante.

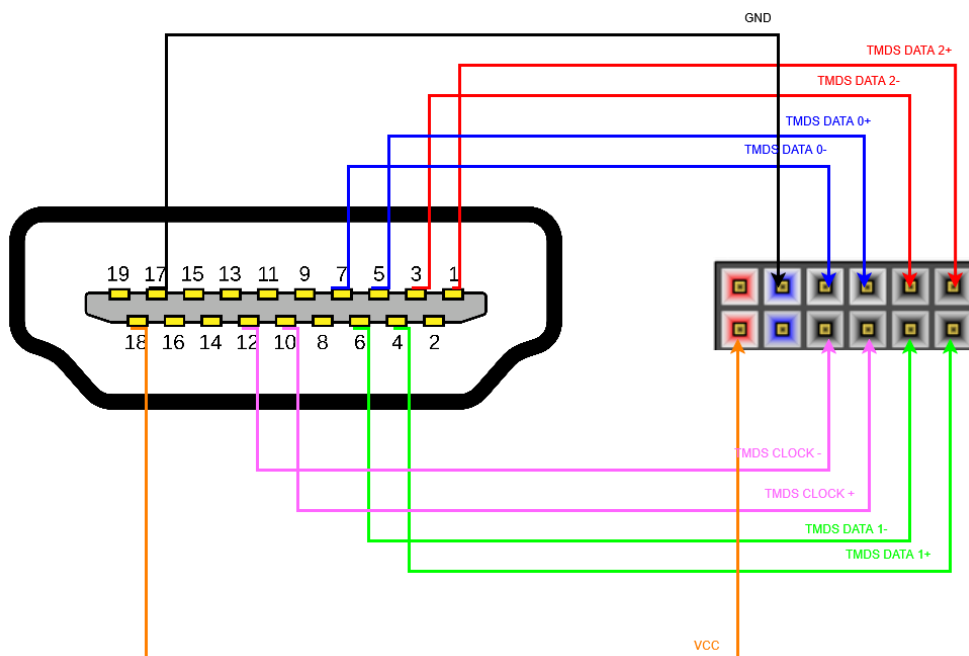


Figura 4.6. Asignación de canales TMDS a los pines del PMOD JB.

4.6. Integración y validación del sistema migrado

Una vez implementadas las adaptaciones anteriores, se procede a integrar el sistema completo sobre la Arty S7-50.

La Figura 4.7 presenta el diseño final en Vivado, donde pueden identificarse los principales componentes de ambos planos funcionales. Las señales de reloj y reinicio se han eliminado para facilitar la lectura del diagrama, al igual que se han diferenciado las interfaces AXI siguiendo el código de colores de la Figura 4.2: AXI4-Lite de color amarillo, AXI4-Stream en rojo y AXI4 en naranja.

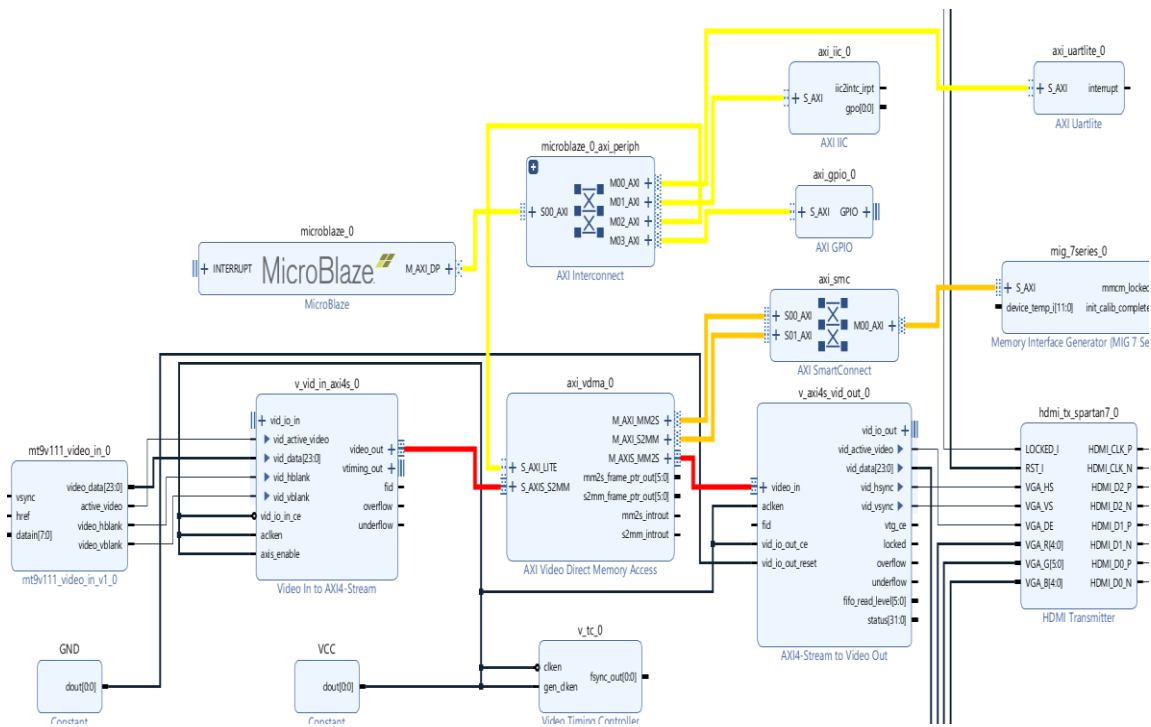


Figura 4.7. Implementación en Vivado del sistema de video migrado.

La integración requirió adaptar la aplicación software original a la nueva arquitectura hardware, incorporando el soporte para los nuevos periféricos implementados y actualizando los parámetros asociados al acceso a memoria. No obstante, la secuencia general de inicialización del sistema de referencia se mantuvo inalterada.

Una vez completada la integración hardware-software, se verificó de forma progresiva el funcionamiento de cada subsistema, incluyendo la comunicación I2C con la cámara, el acceso a la memoria DDR3L a través del controlador MIG y la configuración de los canales del bloque VDMA. Finalmente, se validó el canal completo mediante la visualización en tiempo real de las imágenes capturadas.

La Figura 4.8 muestra el montaje utilizado para realizar las pruebas. Por su parte, la Figura 4.9 presenta un ejemplo de imagen adquirida y visualizada.

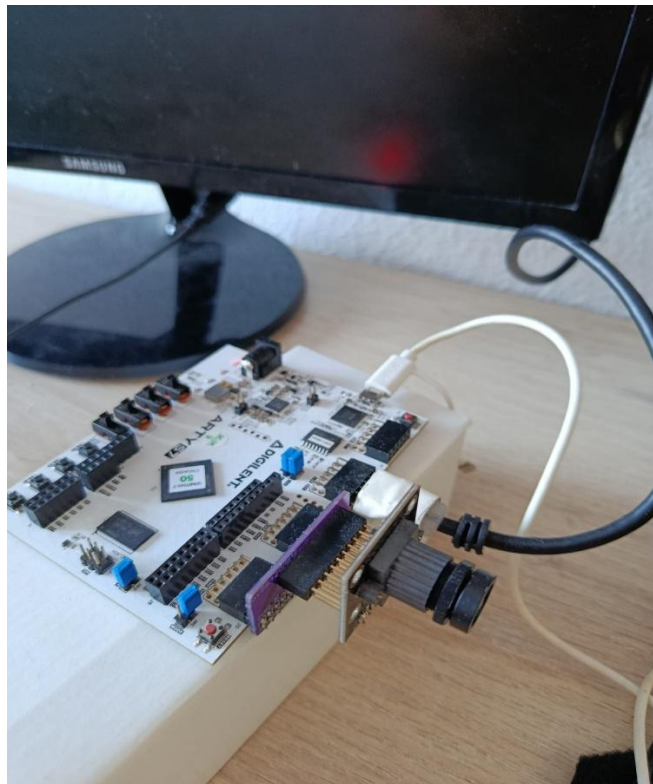


Figura 4.8 Montaje de pruebas. Arty S7-50, cámara y salida HDMI adaptada.

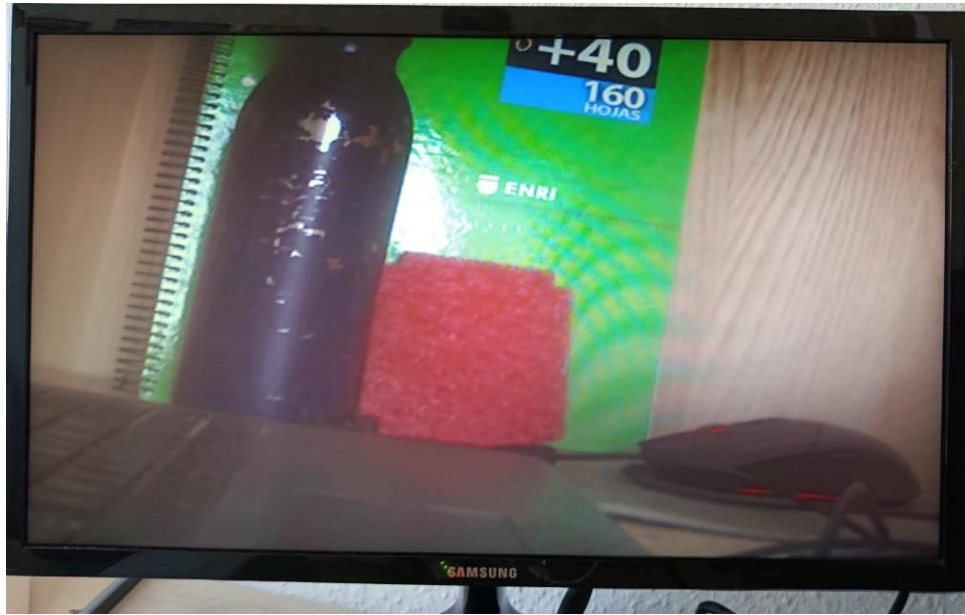


Figura 4.9. Imagen capturada y visualizada en tiempo real.

La correcta reproducción del vídeo confirma que las dependencias identificadas en la Tabla 3.2 han sido sustituidas satisfactoriamente. Sobre esta arquitectura funcional, se desarrollan las ampliaciones del capítulo siguiente.

Capítulo 5. Ampliación del sistema de vídeo

5.1. Objetivos de la ampliación

Una vez obtenido un canal de vídeo funcional sobre la FPGA discreta, el siguiente paso es evaluar la capacidad del sistema migrado para crecer y asumir funciones que el diseño original no contemplaba. Con esta finalidad, se incorporan dos ampliaciones al diseño: un subsistema de monitorización que obtiene métricas de rendimiento en tiempo real, tanto de los accesos a memoria externa como de la ejecución de rutinas software, y un módulo de procesamiento y generación de vídeo diseñado mediante HLS, configurable desde la aplicación software.

Más que por la complejidad de los algoritmos implementados, ambos subsistemas resultan de interés por lo que su integración supone: que el Plano de Datos admite nuevos bloques de procesamiento sin alterar el determinismo del flujo de vídeo, que el Plano de Control puede incorporar nuevos periféricos sin rediseñar la red de interconexión, y que el procesador MicroBlaze, dimensionado al mínimo durante la migración, puede coordinar funciones más complejas que la inicialización del sistema.

Esta última idea es el fundamento principal del capítulo. Ambos subsistemas no operan de forma aislada, sino que interaccionan a través de MicroBlaze. Así, el procesador recopila las métricas adquiridas por la monitorización y las transfiere al módulo HLS, que las representa sobre la propia salida de vídeo. El sistema queda así capacitado para mostrar su rendimiento de forma autónoma, confirmando que la arquitectura migrada constituye una plataforma extensible y no un diseño cerrado.

Los apartados siguientes describen cada extensión propuesta y su integración.

5.2. Sistema de monitorización

El primer subsistema incorporado dota al sistema de la capacidad de medir su propio rendimiento, con un doble propósito. Por un lado, respaldar las soluciones adoptadas durante la migración, contrastando el comportamiento real con el ancho de banda teórico estimado en el apartado 4.3.2. Por otro, servir como herramienta de diagnóstico, al permitir comprobar si las transferencias de memoria ocurren al ritmo previsto cuando la salida de vídeo no es la esperada.

Para implementar dicho subsistema se han integrado dos bloques IP a la red AXI4-Lite existente: el módulo *AXI Performance Monitor (APM)* [14], que supervisa el tráfico a través de interfaces AXI, y el *AXI Timer* [15], que proporciona la referencia temporal para fijar los intervalos de medición.

La implementación de este subsistema mediante hardware dedicado permite desacoplar la adquisición de las medidas de la ejecución del software. Así, MicroBlaze únicamente interviene en la lectura y procesado de los resultados, mientras que la captura se realiza de forma autónoma y en paralelo al funcionamiento del canal.

La Figura 5.1 muestra la incorporación de ambos bloques al sistema. A partir de esta nueva infraestructura, se diseñan dos mecanismos de medida complementarios.

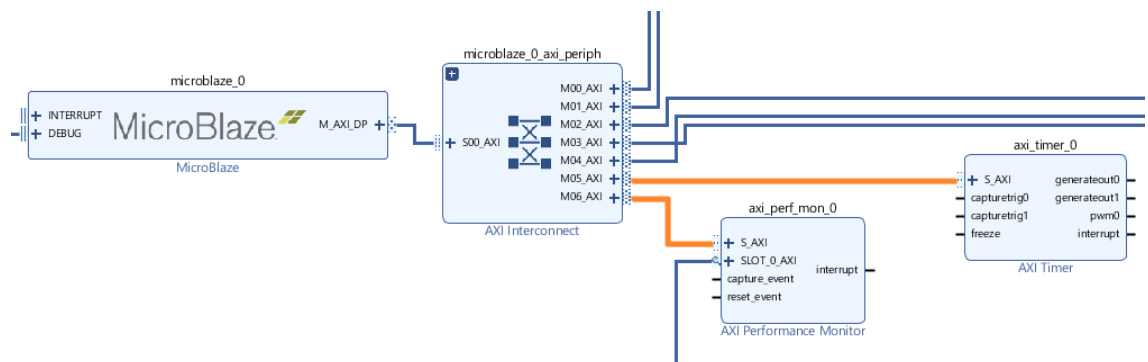


Figura 5.1 Periféricos de monitorización en la red de control (en naranja).

5.2.1. Medida del ancho de banda de memoria

El ancho de banda consumido por la transferencia de fotogramas es la métrica más representativa del rendimiento de un sistema de vídeo, pues refleja si el acceso a memoria funciona al ritmo que exige la captura y visualización.

Para adquirir esta medida, el puerto de monitorización del bloque APM se sitúa sobre la interfaz AXI4 entre el controlador MIG y la estructura AXI SmartConnect. Esta ubicación permite registrar de forma independiente el tráfico de los canales de lectura y escritura del bloque VDMA a través de una misma sonda. La Figura 5.2 muestra la conexión realizada.

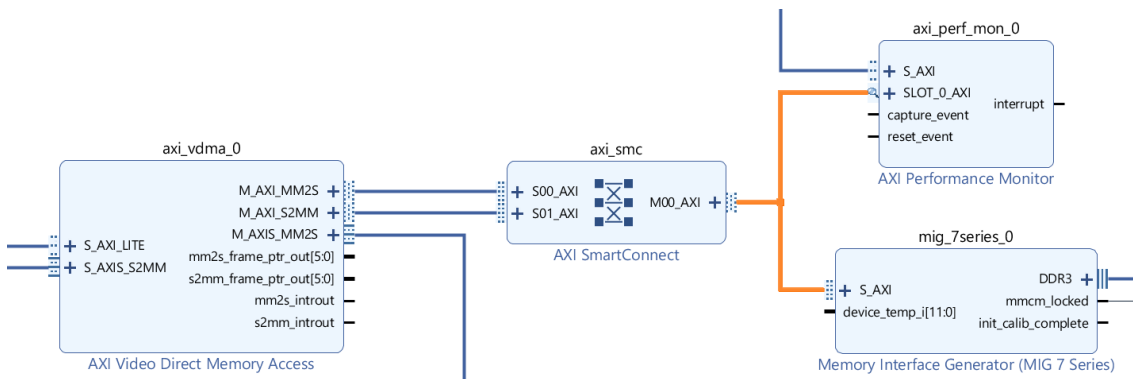


Figura 5.2 Sonda del APM sobre la interfaz AXI4 del MIG (en naranja).

Durante la ejecución, MicroBlaze emplea el *AXI Timer* para definir ventanas de medida de duración fija. En cada intervalo, el APM contabiliza los *bytes* transferidos por uno de los canales y, al finalizar dicho intervalo, el procesador lee el contador y calcula el ancho de banda efectivo mediante:

$$BW = \frac{\text{Bytes transferidos}}{\text{Tiempo de medida}}$$

Ambas métricas se obtienen en ventanas consecutivas. Dado que el sistema de vídeo opera con tasas de transferencia constantes, medir cada canal en un intervalo distinto no afecta a la validez de la comparación.

5.2.2. Medida del tiempo de ejecución software

El segundo mecanismo evalúa el tiempo que tarda MicroBlaze en ejecutar una rutina software. Empleando una rutina concreta como referencia, esta

medida permite comparar el rendimiento de distintas configuraciones del procesador.

Para ello, se emplea el contador del bloque *AXI Timer*, registrando su valor inmediatamente antes y después de la sección de código analizada. La diferencia entre ambas lecturas se corresponde con el número de ciclos transcurridos N_{ciclos} , del que se obtiene el tiempo a partir de la frecuencia del temporizador f_{clk} :

$$t_{ejecución} = \frac{N_{ciclos}}{f_{clk}}$$

Esta medida se realiza bajo demanda sobre rutinas software seleccionadas. Para que esta métrica no interfiera con las ventanas del ancho de banda, el contador opera en modo libre (*free run*) y únicamente se consulta su valor en los instantes necesarios. Las comparativas obtenidas con este mecanismo se estudian en el Capítulo 6.

5.3. Módulo de procesamiento y generación de vídeo mediante HLS

Como segunda ampliación de la arquitectura, se ha diseñado un módulo de procesamiento y generación de vídeo mediante HLS. Su desarrollo permite estudiar cómo incorporar un acelerador hardware al Plano de Datos, respetando el flujo en tiempo real y las estructuras AXI existentes, y cómo hacerlo accesible desde software. Los apartados siguientes abordan su integración en la arquitectura, sus modos de funcionamiento y la representación de las métricas en pantalla.

5.3.1. Arquitectura e integración del módulo

La Figura 5.3 muestra el bloque procesador integrado dentro del sistema. Como se observa, el módulo se inserta en el flujo AXI4-Stream antes del VDMA. Esta ubicación es deliberada pues, en los modos de generación, las imágenes

producidas deben atravesar el mismo mecanismo de *frame buffers* que el vídeo capturado por la cámara. Para integrarse en la cadena de datos, el módulo incorpora interfaces AXI4-Stream de entrada y salida. Además, cuenta con una interfaz AXI4-Lite conectada a la red de control, a través de la cual MicroBlaze selecciona el modo de funcionamiento y transmite el valor de la métrica a representar.

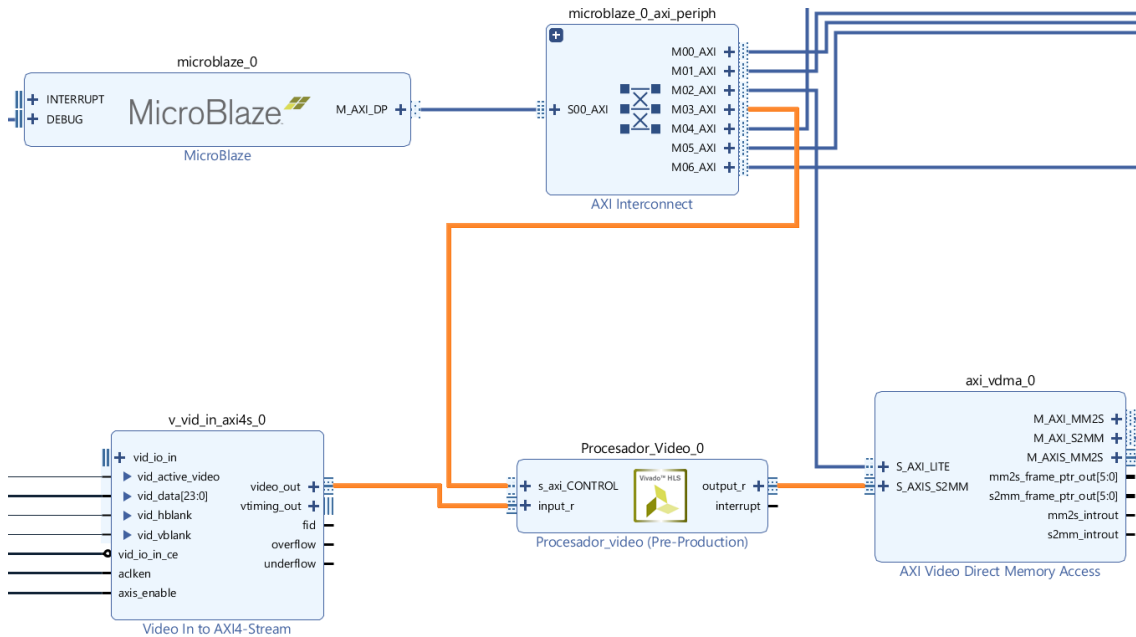


Figura 5.3 Integración del módulo HLS en el flujo de vídeo (conexiones en naranja).

La segmentación del bloque en interfaz de control y datos plantea un conflicto de relojes. En Vitis HLS, la herramienta genera por defecto un único reloj para todo el módulo, lo que obligaría a ambas interfaces a operar a la misma frecuencia. Sin embargo, en el capítulo anterior se estableció que el flujo AXI4-Stream se sincronizaba a 81,25 MHz y la red AXI4-Lite a 50 MHz.

Para respetar esta separación y no añadir lógica extra a la interconexión, se configuró el bloque HLS de manera que la interfaz de control emplea un reloj independiente del flujo de datos. La Figura 5.4 muestra las directivas HLS usadas para generar las interfaces y dicho reloj.

```
// Interfaces de vídeo AXI4-Stream (81,25 MHz)

#pragma HLS INTERFACE axis port=input

#pragma HLS INTERFACE axis port=output

// Interfaz de control AXI4-Lite (50 MHz, reloj
independiente)

#pragma HLS INTERFACE s_axilite port=mode
bundle=CONTROL

#pragma HLS INTERFACE s_axilite port=mbs_entero
bundle=CONTROL

#pragma HLS INTERFACE s_axilite port=mbs_decimal
bundle=CONTROL

#pragma HLS INTERFACE s_axilite port=camara_activa
bundle=CONTROL

#pragma HLS INTERFACE s_axilite port=return
bundle=CONTROL clock=control_clk
```

Figura 5.4 Directivas HLS de interfaces y reloj de control.

5.3.2. Modos de funcionamiento

En lugar de desarrollar aceleradores independientes para cada tarea, se ha optado por un único bloque configurable cuya función se selecciona dinámicamente mediante un registro de control. Esta estrategia reutiliza el mismo circuito hardware para operaciones distintas, reduciendo la ocupación de recursos. La Tabla 5.1 resume los modos implementados.

Los modos de procesamiento operan sobre el flujo de la cámara, siendo un modo de paso directo (*bypass*), que reenvía el vídeo sin procesar, y un modo de

conversión a escala de grises. Este último implica una decisión orientada a su implementación sobre el hardware.

La conversión de color en formato RGB a escala de grises se suele modelar mediante la ecuación de luminosidad estándar:

$$Gris = (R \cdot 0,299) + (G \cdot 0,587) + (B \cdot 0,114)$$

Implementar esta expresión de forma directa con aritmética en punto flotante implicaría dedicar múltiples bloques DSP (*Digital Signal Processing*) como multiplicadores dedicados y una considerable lógica adicional. En su lugar, se emplea una aproximación basada en operaciones con enteros que reproduce el mismo resultado visual:

$$Gris = \frac{(R \cdot 76) + (G \cdot 150) + (B \cdot 30)}{256}$$

En el Capítulo 6 se comparan distintas formas de implementar esta conversión y cuantifica el ahorro frente a la versión en punto flotante.

Los modos de generación producen la imagen internamente, sin leer la entrada, lo que permite validar la salida de vídeo en ausencia de cámara. El bloque recorre el fotograma mediante dos contadores, uno para las filas y otro para las columnas, creando la salida píxel a píxel en función del algoritmo elegido. Estos comprenden un patrón fijo de barras de color y un patrón dinámico, consistente en una figura que se desplaza y rebota en los bordes del cuadro.

El desacoplamiento de la entrada en estos modos tiene una consecuencia relevante. Al generar un píxel por ciclo a 81,25 MHz, el módulo escribe a una tasa de ~244 MB/s, muy superior a la que se tiene con la cámara. Esto convierte a los modos de generación en una prueba de estrés para el subsistema de memoria, cuyo comportamiento bajo esta carga se analiza en el Capítulo 6. El desacoplamiento también afecta al patrón dinámico. Como el bloque genera fotogramas más rápido que la salida los muestra, un contador actualiza la posición de la figura una vez cada período de 60 Hz, de modo que su movimiento se perciba continuo.

Modo	Tipo	Funcionamiento
0	Procesado	Paso directo del vídeo de la cámara
1	Procesado	Conversión a escala de grises
2	Generación	Patrón de barras fijas de colores
3	Generación	Patrón de figura en movimiento
4	Generación/Automático	Patrón de ausencia de señal

Tabla 5.1. Modos de funcionamiento del módulo HLS.

Los modos de procesado, por su parte, requieren una salvaguarda. Si se seleccionara uno de ellos sin estar la cámara conectada o activa, el módulo intentaría leer un píxel de entrada que nunca llega, bloqueando la cadena de vídeo.

Para evitar esta problemática, MicroBlaze comunica al bloque, mediante un registro de control, si la cámara se encuentra operativa tras su inicialización. El módulo consulta dicho registro antes de actuar: si se solicita un modo de procesado con la cámara activa, procede con normalidad. En cambio, cuando se pide un modo de procesado con el sensor no disponible, el módulo deriva automáticamente la salida a un patrón de ausencia de señal, evitando una lectura bloqueante y señalando visualmente la ausencia de entrada. El flujograma de la Figura 5.5 representa el funcionamiento de este mecanismo de salvaguarda.

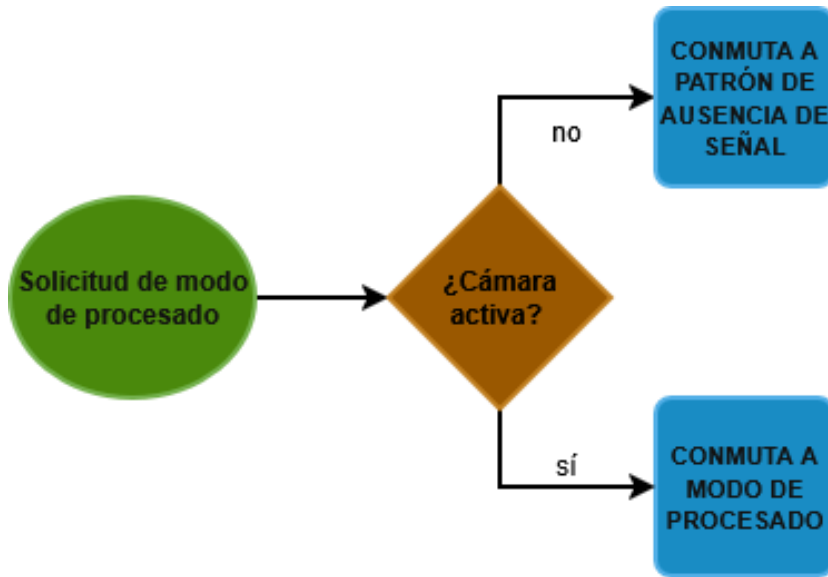


Figura 5.5 Mecanismo de salvaguarda.

Por último, con independencia del modo, el módulo debe operar a un píxel por ciclo del canal de vídeo. Para ello, el algoritmo de procesado se segmenta con un intervalo de iniciación unitario ($II = 1$) mediante una directiva HLS.

5.3.3. Superposición de métricas en pantalla (OSD)

Para cualquier modo activo, el módulo superpone sobre el vídeo de salida una capa de información en pantalla (*On-Screen Display, OSD*) que representa el ancho de banda total adquirido por el sistema de monitorización.

MicroBlaze transmite este valor al bloque de procesamiento a través de la interfaz AXI4-Lite, descompuesto en parte entera y decimal para facilitar su representación. Al comienzo de cada fotograma, el módulo separa la cifra en sus dígitos y obtiene la representación de cada carácter de una memoria de fuente alojada en el propio bloque. Después, durante el barrido del cuadro, compone la métrica píxel a píxel: cuando los contadores internos de fila y columna alcanzan la región reservada en la esquina superior izquierda, el módulo dibuja el carácter correspondiente. La Figura 5.6 muestra la posición y las dimensiones de la región de dibujado dentro del cuadro de vídeo.

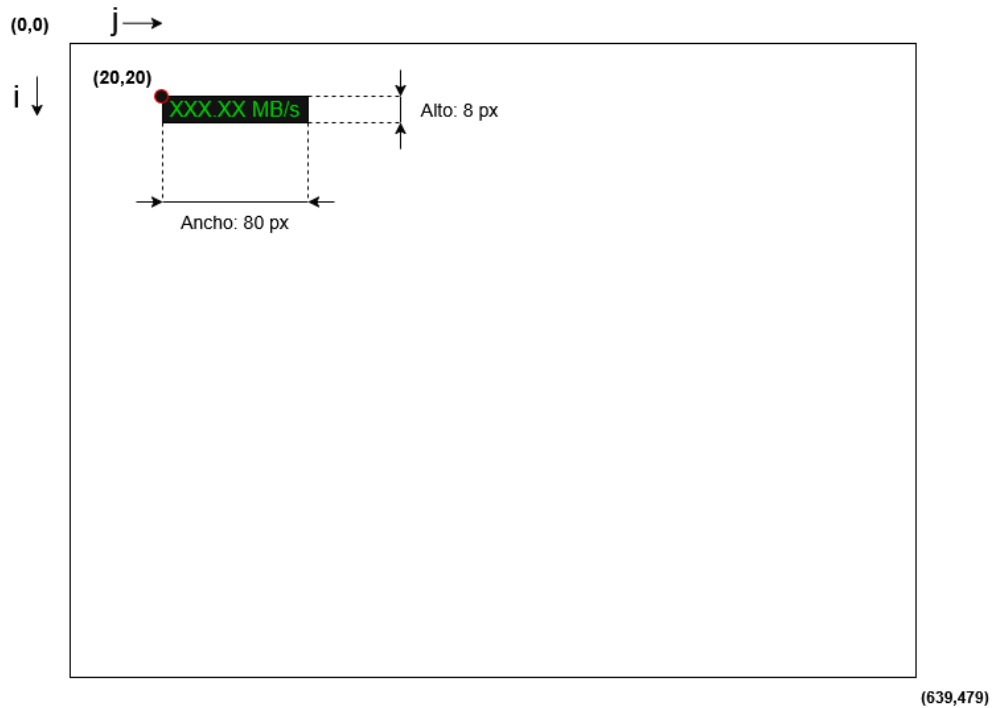


Figura 5.6 Esquema de representación del OSD en un fotograma.

El OSD es el punto donde convergen las dos ampliaciones propuestas: dos subsistemas hardware, la monitorización y el procesador de vídeo, interaccionan a través de MicroBlaze de forma autónoma. Es la manifestación de la flexibilidad que persigue este capítulo. La Figura 5.7 ilustra el esquema de interacción hardware-software descrito.

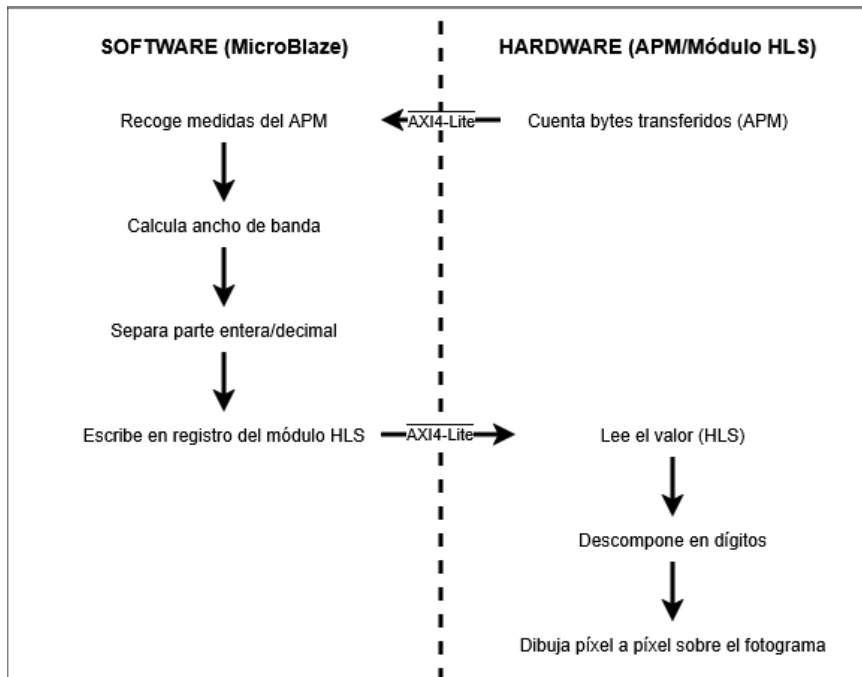


Figura 5.7 Interacción hardware-software para la representación del OSD.

5.3.4. Sincronización del OSD con el flujo de vídeo

Los fotogramas se sincronizan correctamente a la salida gracias a las señales de alineación implícitas en el protocolo AXI4-Stream, entre ellas, las que indican el comienzo y final de la imagen. Esta sincronización no se pierde al conmutar entre modos, pero sí afecta a la posición del OSD.

Como se ha visto en la Figura 5.6, el OSD tiene una representación fija en el fotograma definida por los contadores internos de fila y columna que indican la posición del cuadro en cada instante. En los modos de generación, la imagen se genera a partir de estos mismos contadores, por lo que la alineación del cuadro de vídeo tiene el mismo origen que el OSD.

En los modos de procesado, en cambio, el sincronismo del cuadro viene impuesto por la captura de la cámara, mientras que los contadores internos podrían arrastrar un desfase previo: si los contadores no arrancan alineados con la marca, el módulo cree estar en el píxel (0,0) cuando la cámara entrega un píxel cualquiera, y el OSD aparece desplazado sobre el fotograma.

Para mantener la alineación, los modos de procesado anteponen al tratamiento de la imagen una etapa de resincronización que se ejecuta siempre al comienzo de cada fotograma. Su funcionamiento es simple pero efectivo: el módulo lee los píxeles del flujo descartándolos hasta encontrar la marca de inicio de fotograma, momento en el cual arranca sus contadores y procede con el procesado, ya alineado con el sensor.

La clave de este mecanismo es que no necesita detectar cuándo se ha perdido la sincronía, sino que actúa de forma incondicional siempre que se está en un modo de procesado. Sin embargo, su coste únicamente repercute al primer fotograma de transición: una vez se han alineado los contadores, el primer píxel leído al comienzo del fotograma es la propia marca de inicio, y la etapa concluye en la primera iteración sin descartar nada. Solo cuando existe un desfase real, típicamente tras llegar desde un modo de generación o un reinicio de la cámara, la etapa descarta píxeles hasta la siguiente marca.

De este modo, sin lógica de detección de eventos, el mecanismo garantiza que el procesado siempre comience en el inicio del cuadro, manteniendo el OSD

en su posición con coste nulo en régimen estable. La Figura 5.8 ilustra su funcionamiento.

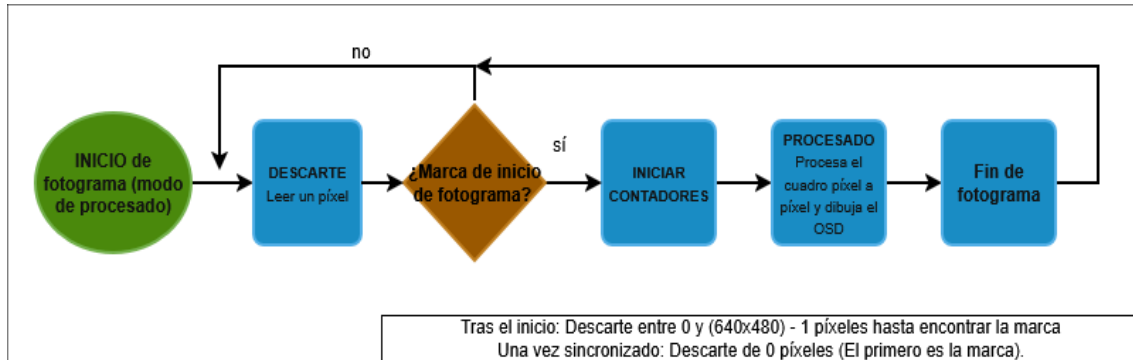


Figura 5.8 Flujo de la etapa de resincronización del OSD en los modos de procesado.

5.4. Arquitectura del sistema de vídeo completo

La Figura 5.9 presenta el sistema completo, con las dos ampliaciones incorporadas sobre el canal de vídeo básico. En ella se identifican el subsistema de monitorización en el Plano de Control y el módulo de procesamiento en el Plano de Datos, junto al resto del sistema.

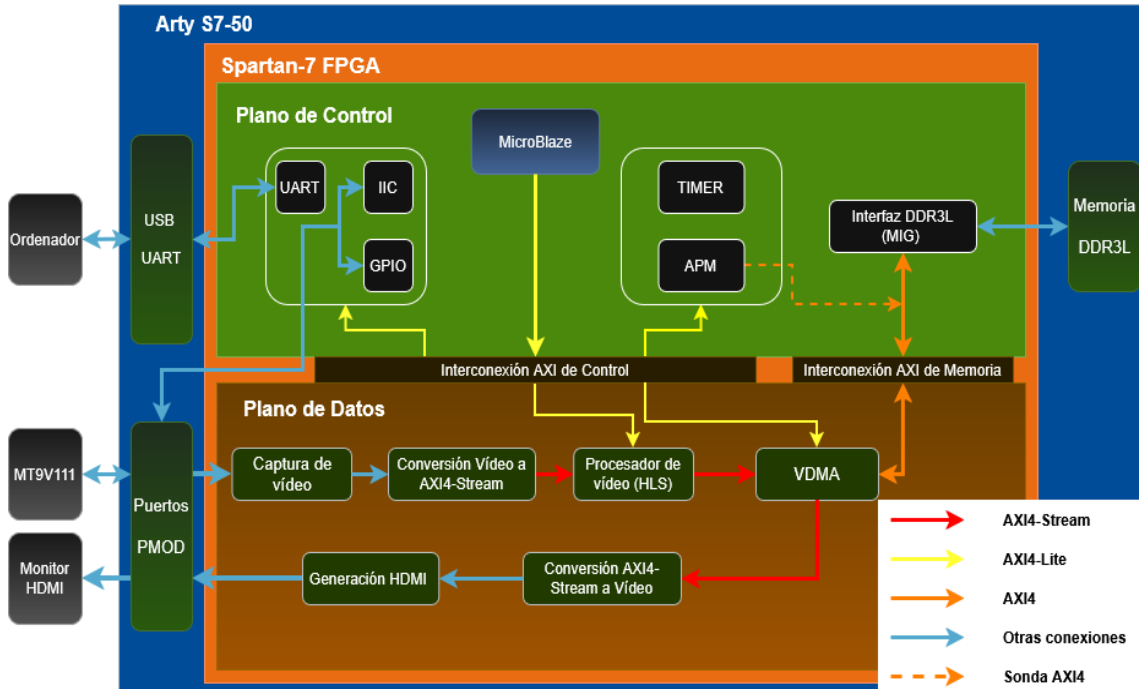


Figura 5.9 Sistema de vídeo completo.

Como se ha podido comprobar, la arquitectura migrada ha acogido ambos subsistemas sin necesidad de rediseño alguno, incorporándolos a las redes AXI

y los dominios de reloj ya existentes. Esta capacidad de expandirse sobre la infraestructura disponible, sin alterar su organización ni limitar el funcionamiento, es la confirmación práctica de la flexibilidad que se planteaba en el inicio del capítulo.

5.5. Aplicación software de control final

La incorporación de los nuevos bloques modifica el papel del software. Frente a la secuencia de inicialización del sistema vista en el apartado 3.3, donde el procesador configuraba los periféricos, arrancaba la cámara y cedía la mayor parte del control al hardware, la aplicación ampliada asume un rol activo y permanente. Tras la inicialización, atiende las órdenes del usuario y gestiona la adquisición y cálculo de métricas a lo largo de toda la ejecución. La Figura 5.10 representa el nuevo paradigma de control.

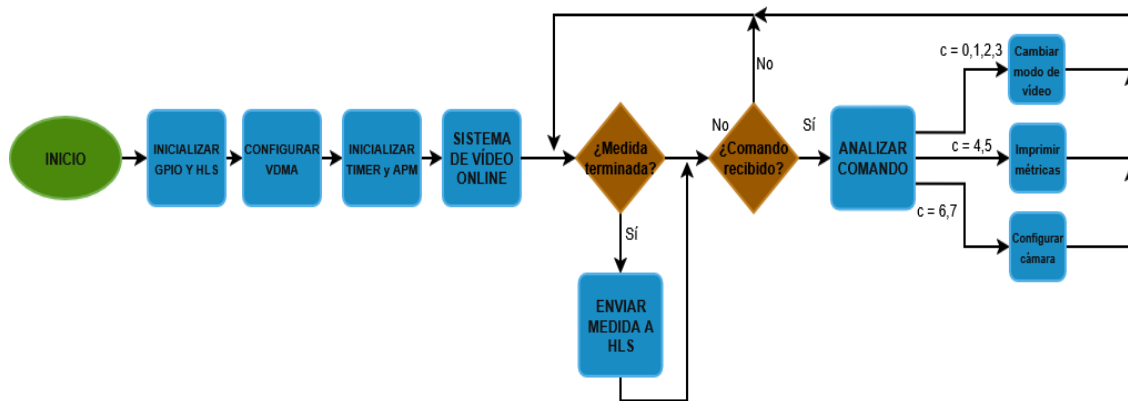


Figura 5.10 Diagrama de flujo de la aplicación software final.

La interacción se realiza mediante un menú por terminal UART. El usuario introduce un comando numérico y el software ejecuta la acción asociada. Las opciones, recogidas en la Tabla 5.2, se reparten entre la selección del modo de vídeo, que se traduce a una escritura en el registro de configuración del módulo HLS y los comandos de monitorización y sistema, que actúan sobre los datos de medida, la cámara o el propio procesador.

Comando	Acción
0	Modo de vídeo: paso directo
1	Modo de vídeo: escala de grises
2	Modo de vídeo: patrón de barras
3	Modo de vídeo: patrón dinámico
4	Imprimir el desglose de ancho de banda (total, lectura y escritura)
5	Imprimir el tiempo de ejecución de la rutina del comando 4
6	Arranque de la cámara
7	Reinicio de la cámara

Tabla 5.2. Comandos de la aplicación de control.

En contraste con el sistema de referencia, la aplicación final no inicializa la cámara durante el arranque, sino que la activa bajo demanda mediante el comando correspondiente. Al recibirlo, se configura el sensor por I2C y, en caso de éxito, notifica al módulo que el sensor está operativo, habilitando los modos de procesado. Hasta entonces, únicamente los modos de generación estarán activos, y se conmutará al patrón de ausencia de cámara en caso de elegir un modo de procesado.

En cuanto al subsistema de monitorización, a la rutina descrita en el apartado 5.2.1 se le añade que, tras calcular los anchos de banda, MicroBlaze transfiere su suma total al bloque HLS para su representación mediante el OSD. Cabe resaltar que el comando para la impresión de esta medida mostrado en la Tabla 5.2 no inicia la medición, sino que imprime por el terminal el desglose detallado de los últimos datos recopilados. El comando de medida temporal, por su parte, aplica el mecanismo descrito en el apartado 5.2.2 sobre la rutina de impresión de ancho de banda.

Con la aplicación de control definida, el sistema de vídeo queda completo: la migración del Capítulo 4 y las ampliaciones de este capítulo conforman la arquitectura final propuesta.

Capítulo 6. Resultados y validación

En este capítulo se evalúa el sistema en tres áreas diferentes. En primer lugar, se validará su funcionamiento de forma cualitativa y cuantitativa. Después, se presentarán los costes de su implementación en recursos y potencia consumida, y se justificarán con valores objetivos las principales decisiones de diseño tomadas durante el desarrollo. Por último, el canal de vídeo básico migrado se someterá a una comparativa con el sistema de referencia.

Las medidas proceden tanto del subsistema de monitorización descrito en el apartado anterior, como de mediciones experimentales y de los informes de implementación de Vivado.

6.1. Validación funcional

La validación se ha realizado sobre el montaje completo del Capítulo 4: Arty S7-50, cámara MT9V111 y salida HDMI adaptada. La Figura 6.1 muestra los mensajes de depuración que confirman la correcta inicialización del sistema, contrastable con el flujograma de la Figura 5.10.

```

Serial: (COM9, 9600, 8, 1, None, None - CONNECTED) - Encoding: (ISO-8859-1)
--- Entrando main() ---

-----
Bloque GPIO iniciado

Bloque HLS iniciado

--- Inicializando canales ...

    Configurando canal de escritura ...
Reiniciando Canal de Escritura VDMA ...
Estableciendo canal de Escritura del VDMA ...
    vdma_setup: Dirección 0 = 0x81000000.
    vdma_setup: Dirección 1 = 0x810E1000.
    vdma_setup: Dirección 2 = 0x811C2000.
VDMA - Iniciando canal de Escritura ...

--- Configurando canal de lectura ...
Reiniciando Canal de Lectura VDMA ...
Estableciendo canal de Lectura del VDMA ...
    vdma_setup: Dirección 0 = 0x81000000.
    vdma_setup: Dirección 1 = 0x810E1000.
    vdma_setup: Dirección 2 = 0x811C2000.
VDMA - Iniciando canal de Lectura ...

Bloque APM iniciado

Bloque Timer iniciado

--- Sistema de Procesado de Video Online ---
Seleccione una opción:
0: Modo de vídeo : Bypass
1: Modo de vídeo : Escala de Grises
2: Modo de vídeo : Test Pattern Barras Fijas
3: Modo de vídeo : Test Pattern en movimiento
4: Mostrar ancho de banda de DDR3L
5: Mostrar tiempo de ejecución software de 4
6: Iniciar la cámara MT9V111
7: Reiniciar la cámara

```



Figura 6.1 Mensajes de inicialización del sistema por terminal serie.

Las Figuras 6.2 a 6.5 muestran la salida de vídeo para los distintos modos, con el OSD correctamente superpuesto y mostrando el ancho de banda total en tiempo real. Se observa que, en los modos de generación, el sistema opera en ausencia de la cámara.

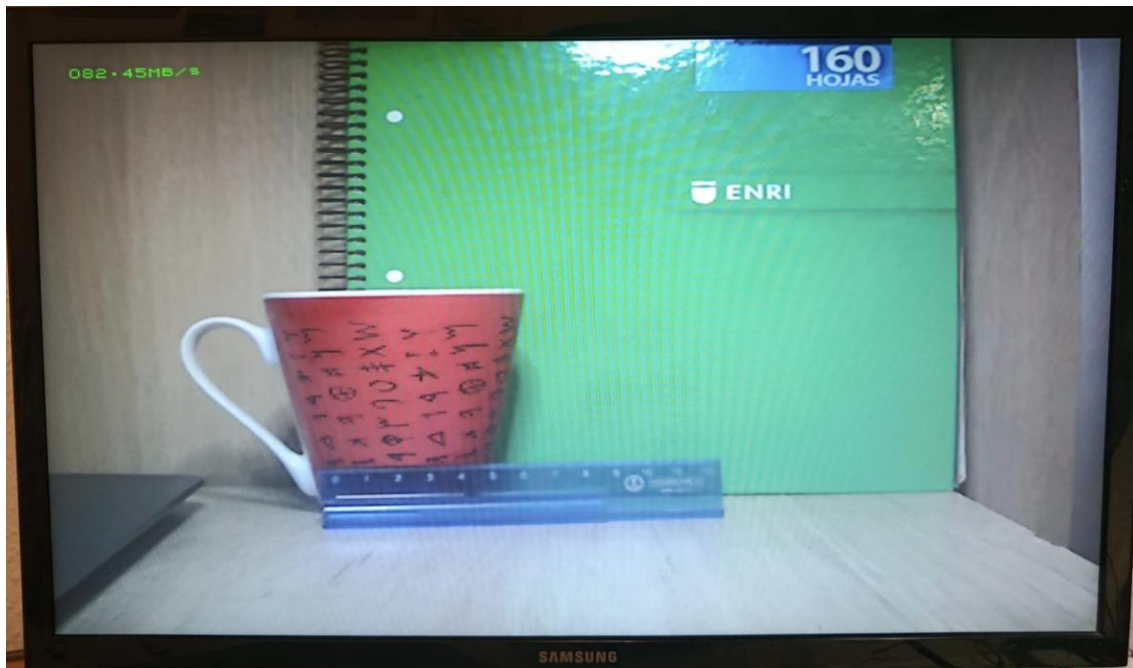


Figura 6.2 Salida de vídeo en modo de paso directo.



Figura 6.3 Salida de vídeo en modo de escala de grises.



Figura 6.4 Modo de generación: patrón de barras de color.



Figura 6.5 Modo de generación: patrón dinámico

De igual forma, se ha verificado la correcta conmutación entre modos de procesado y generación, comprobando el mecanismo de sincronía del OSD descrito en el apartado 5.3.4. Las Figuras 6.6 y 6.7 muestran su ubicación estable antes y después de un cambio de modo. El temporizador presente en ambas representa la continuidad temporal entre ambas figuras.



Figura 6.6 OSD alineado antes de la conmutación de modo.



Figura 6.7 OSD alineado tras la conmutación de modo.

La Figura 6.8 muestra el patrón de ausencia de señal en caso de conmutar a un modo de procesado sin la cámara activada. Se muestra la plataforma Arty S7-50 con el sensor desconectado.

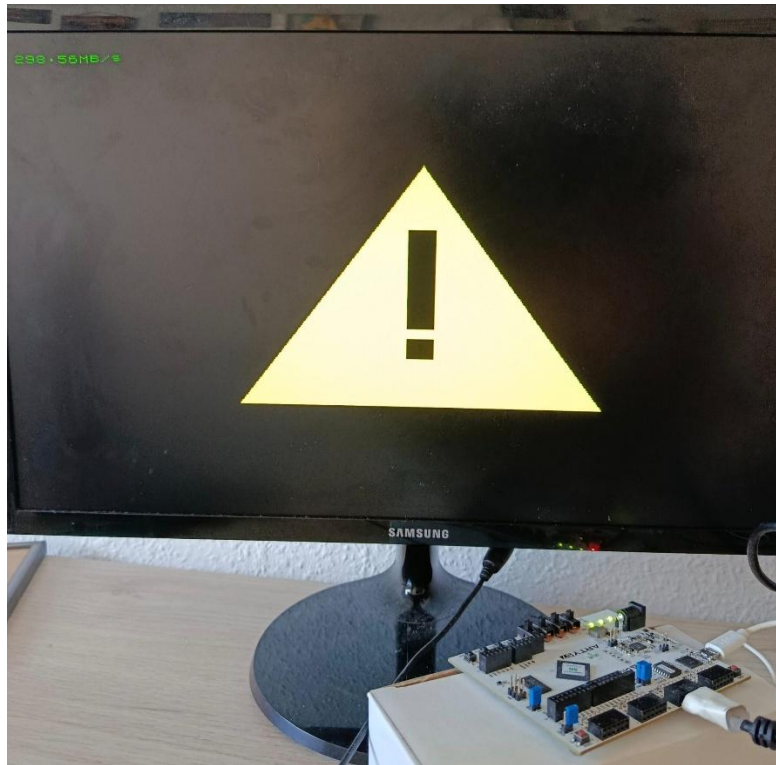


Figura 6.8 Patrón de ausencia de señal.

Las Figuras 6.9 y 6.10 recogen el desglose de una medida de ancho de banda y de tiempo de ejecución software respectivamente.

```
-----
Modo: Bypass activado

Mostrando rendimiento de la DDR3

--- Métrica de Rendimiento DDR3 ---
Tráfico detectado: 41226624 bytes
Ancho de banda lectura: 54.82 MB/s
Ancho de banda escritura: 27.62 MB/s
Ancho de banda total: 82.45 MB/s
Uso del bus (sobre 1300 MB/s): 6.34%
-----
```

Figura 6.9 Medida de ancho de banda en caso de procesado de vídeo.

```

--- Tiempo de ejecución de MicroBlaze ---
Latencia en ciclos del temporizador (50 MHz) : 3816
Tiempo de ejecución total: 0.076 ms
-----

```

Figura 6.10 Tiempo de ejecución de MicroBlaze medido.

6.2. Ancho de banda de memoria

El subsistema de monitorización permite contrastar el ancho de banda real con la estimación teórica del apartado 4.3.2, que preveía 55,3 MB/s y 27,65 MB/s para la salida de vídeo a 60 fps y la entrada a 30 fps, respectivamente. La Tabla 6.1 recoge las medidas, obtenidas mediante la rutina de la Figura 6.9, de un caso capturando vídeo desde la cámara y otro de generación interna.

Escenario	Lectura (MB/s)	Escritura (MB/s)	Total (MB/s)	Uso del bus (% sobre 1300 MB/s)
Vídeo desde cámara	54,82	27,62	82,44	6,3 %
Vídeo generado internamente	54,82	243,70	298,52	22,9 %

Tabla 6.1 Ancho de banda de memoria medido en procesado y generación de vídeo.

Operando con la cámara, las medidas no difieren más del 1% de la predicción teórica. Expresándolas en fotogramas, equivalen a 29,97 fps de captura y 59,48 fps de visualización, siendo muy próximos a los valores esperados. La pequeña desviación es atribuible a que las frecuencias de píxel

generadas en la placa no coinciden de forma exacta con las nominales, como se indicó en el Capítulo 4. El canal opera, por tanto, a la tasa prevista consumiendo una fracción minúscula de la capacidad del controlador de memoria.

En modo de generación, la tasa de escritura se eleva a 264 fps, casi nueve veces más que la captura. Aun así, el sistema mantiene la transferencia sin errores y el uso del bus no alcanza ni un cuarto de su capacidad. Estos resultados confirman que resolver el acceso a memoria externa mediante el MIG, en lugar del controlador integrado de la plataforma de referencia, no supone compromiso alguno de rendimiento.

6.3. Utilización de recursos del sistema migrado

El objetivo principal de este proyecto era reproducir sobre una FPGA discreta el canal de vídeo de referencia. Este apartado cuantifica los costes de la migración en recursos lógicos.

La Tabla 6.2 resume la utilización del canal de vídeo migrado, dándose una representación porcentual respecto a los recursos lógicos totales de la FPGA XC7S50, consultado en [9].

Recurso	Utilizados	Disponibles	Ocupación
LUT (<i>Look Up Table</i>)	12439	32600	38,2%
FF (<i>Flip-Flop</i>)	14655	65200	22,5%
BRAM	23	75	30,7%
DSP	0	120	0%

Tabla 6.2 Utilización de recursos lógicos del sistema migrado.

El sistema ocupa en torno a un tercio de los recursos totales del chip, dejando un amplio margen para ampliaciones. Sin embargo, esta cifra global no responde a la cuestión principal: qué fracción de esos recursos se corresponde

con la infraestructura que sustituye al Sistema de Procesamiento. La Tabla 6.3 desglosa el coste del Plano de Control.

Bloque	LUT	FF	BRAM
Controlador de memoria (MIG)	4934	4001	0
Interconexiones (InterConnect + SmartConnect)	2257	3637	0
Procesador MicroBlaze	1229	1059	16
Periféricos (UART + IIC)	466	463	0
Total Plano de Control	8886	9160	16

Tabla 6.3 Desglose de recursos de la infraestructura de control.

Las adaptaciones realizadas consumen más de 8800 LUT, alrededor del 70% del sistema migrado. El controlador de memoria es con diferencia el más costoso, seguido de las interconexiones. El procesador y los periféricos ocupan una fracción menor del total, coherente con el dimensionado mínimo adoptado en el Capítulo 4. Las ~3600 LUT restantes se corresponden al canal de vídeo propiamente dicho, que existiría con independencia de la plataforma.

La ocupación global tiene una conclusión relevante en cuanto a portabilidad. El sistema migrado diseñado, más allá de en la Arty S7-50, podría implementarse en un dispositivo más modesto como la FPGA XC7S25, con unas 14600 LUT según [1]. Aunque tendría un margen muy ajustado, la posibilidad teórica existe. Que ese margen exista se debe, en buena medida, a las decisiones de diseño tomadas durante la migración, que se contrastan a continuación.

6.3.1. Decisiones de diseño de la migración

Cada una de las siguientes comparativas aísla una variable de diseño, manteniendo el resto del sistema inalterado, para cuantificar el impacto de la decisión adoptada frente a su alternativa. El análisis se centra en las dos decisiones de mayor efecto sobre los recursos.

Infraestructura de interconexión

En el apartado 4.4 se optó por segmentar la infraestructura de interconexión en dos redes independientes para el Plano de Control y los accesos a memoria externa, en dominios de reloj separados. La alternativa analizada es una única interconexión que reúne a todos los maestros y esclavos. La Tabla 6.4 compara ambas configuraciones.

Infraestructura	LUT	FF
Redes segmentadas (adoptada)	2257	3637
Interconexión única	6107	7346

Tabla 6.4 Coste de la infraestructura de interconexión.

Unir todo el tráfico de las interfaces AXI en una única interconexión supone casi triplicar el área de la red. La causa se encuentra en que la interconexión debe añadir lógica de sincronización para gestionar los distintos dominios de reloj y adaptar los cruces entre protocolos, como se adelantaba en el Capítulo 4. La segmentación adoptada evita este sobrecoste al confinar cada red a su dominio.

Procesador MicroBlaze

En el apartado 4.3.1 se justificó el uso de un procesador MicroBlaze de configuración ajustada. Como alternativa, se estudia un procesador que incluye cachés de datos e instrucciones y extensiones de cálculo. La Tabla 6.5 compara

ambas configuraciones. El tiempo de ejecución del procesador se ha medido a partir de la rutina de ancho de banda.

Configuración	LUT	FF	BRAM	DSP	Tiempo de ejecución (µs)
Ajustada (adoptada)	1229	1059	16	0	76,32
Cachés y extensiones	2625	2152	22	5	11,96

Tabla 6.5 Coste y rendimiento de la configuración de MicroBlaze.

La versión extendida ofrece un rendimiento siete veces superior para la rutina analizada, pero supone la duplicación de los recursos lógicos ocupados y requiere más memoria BRAM para las cachés y bloques DSP.

De cara al papel de MicroBlaze en el sistema, esa aceleración no aporta ventaja perceptible, ya que el procesador no atiende eventos en tiempo real, sino tareas puntuales y espaciadas que consulta por sondeo, donde el tiempo de respuesta no es crítico. No justifica, por tanto, el área adicional.

La configuración ajustada mantiene el balance entre área y tiempo de procesado requeridos por el canal de vídeo, reservando los recursos lógicos para el canal de vídeo.

Discusión sobre portabilidad

El análisis de recursos sugería que el canal de vídeo básico podría alojarse en dispositivos de menor capacidad que el XC7S50. Para comprobarlo de forma fehaciente, se implementó en Vivado el sistema migrado sobre el dispositivo XC7S25.

La implementación se completó con éxito. El sistema ocupa 12200 LUT, 14500 FF y 23 BRAM sobre el XC7S25, lo que representa en torno al 82 % de sus LUT disponibles. El diseño, por tanto, cabe íntegramente en un

notablemente inferior al XC7S50 y más barato, como se muestra en la Tabla 6.13.

Cabe resaltar que esta implementación es posible debido a las decisiones de diseño que se han tomado. El canal de vídeo básico deja unas 2160 LUT de margen en el chip XC7S25. Una configuración desajustada de MicroBlaze consumiría prácticamente todo el margen, y el sobre coste de una única interconexión lo agotaría por su cuenta. Las decisiones de diseño no son, por tanto, un detalle menor, sino lo que hace el diseño portable a plataformas más modestas.

Conviene precisar el alcance de esta comprobación. Se trata de una implementación completa que confirma que el diseño cabe sobre el XC7S25 cumpliendo las restricciones temporales, no de una validación funcional sobre una placa física, que requeriría el hardware correspondiente. No obstante, la ocupación del 82 % deja un margen estrecho: el dispositivo aloja el sistema actual, pero su capacidad de crecimiento es limitada, lo que lo sitúa como opción idónea para el canal de vídeo básico más que para ampliaciones sustanciales. Esta observación enlaza con el argumento del margen desarrollado en el apartado 6.6: el XC7S25 minimiza el coste para el sistema tal cual es, mientras que el XC7S50 ofrece margen para ampliarlo.

6.4. Coste del sistema ampliado

La Tabla 6.6 recoge la utilización del sistema completo tras añadir las ampliaciones del Capítulo 5, y su incremento respecto al migrado.

Recurso	Migrado	Ampliado	Incremento	Ocupación (Ampliado)
LUT	12439	16658	+4219	51,1 %
FF	14655	19567	+4912	30 %
BRAM	23	23,5	+0,5	31,3 %
DSP	0	15	+15	12,5 %

Tabla 6.6 Recursos del sistema migrado y completo, con el incremento de las ampliaciones.

Las ampliaciones suponen en torno a 4000 LUT adicionales y el uso de bloques DSP, asociados al módulo HLS. El sistema completo alcanza la mitad de ocupación de todo el dispositivo, lo que confirma que la arquitectura migrada acoge las nuevas funciones sin comprometer la holgura de la FPGA.

Las distintas decisiones de diseño tomadas durante el Capítulo 5 se reflejan en la utilización de las ampliaciones. El hecho de hacer un bloque HLS reconfigurable por software reduce su huella en recursos lógicos, al igual que emplear una única sonda del módulo APM para medir todo el tráfico.

Implementación de la conversión a escala de grises

El diseño de los algoritmos de procesamiento también tiene una repercusión directa en el área ocupada. En el apartado 5.3.2 se discutían las distintas formas de implementar la conversión a gris, partiendo de la fórmula estándar y realizando una aproximación del cálculo. La Tabla 6.7 compara las dos implementaciones del módulo completo, variando únicamente el algoritmo de conversión.

Implementación	LUT	FF	DSP	Frecuencia máxima (MHz)
Punto flotante	4837	5570	45	90,57
Entera con multiplicaciones (adoptada)	2393	2385	15	96,61

Tabla 6.7 Coste de distintas implementaciones de la escala a gris (módulo completo).

La aritmética en punto flotante triplica los bloques DSP y duplica las LUT, consumiendo casi el 40% de los procesadores digitales dedicados del dispositivo en una simple conversión. Este sobrecoste no se debe únicamente a que las operaciones en punto flotante sean más costosas que las enteras, sino a su interacción con la restricción de procesamiento en tiempo real: para sostener un intervalo de iniciación unitario ($II=1$) con operaciones de varios ciclos de latencia, la síntesis de alto nivel debe replicar y segmentar el hardware de cálculo, lo que dispara la ocupación.

Por su parte, la formulación entera implementa el mismo funcionamiento con una fracción de los recursos y una menor latencia. La aproximación de los coeficientes de la fórmula reduce notablemente el uso de operadores dedicados y da un resultado efectivo, como se muestra en la Figura 6.3.

6.5. Potencia

La caracterización del consumo se aborda contrastando la estimación de la herramienta de síntesis con la medida directa sobre la plataforma, que constituye la referencia principal por reflejar el comportamiento real del sistema.

Estimación de la herramienta

Al completar la implementación, Vivado proporciona una estimación de la potencia consumida por el circuito desarrollado sobre la FPGA. La Tabla 6.8 recoge las estimaciones para ambos sistemas.

Sistema	Estática (W)	Dinámica (W)	Total (W)
Canal básico (migrado)	0,077	1,030	1,107
Canal completo (ampliado)	0,077	1,052	1,129

Tabla 6.8 Estimación de consumo de Vivado sobre la Arty S7-50.

Esta estimación, sin embargo, debe tomarse con reservas. La herramienta calcula el consumo dinámico a partir de suposiciones sobre la actividad de conmutación de las señales, que no conoce realmente al carecer de información sobre los datos que el sistema procesa en funcionamiento. En diseños complejos, esta limitación produce desviaciones notables respecto al consumo real, una discrepancia que se ha notificado en otros trabajos [17]. Este problema también se manifiesta en la estimación de potencia consumida por los procesadores dedicados, como se discute en el apartado 6.6.

Además, la estimación se limita a la lógica implementada en la FPGA y no contempla el resto de los elementos de la placa, por lo que no caracteriza el sistema como aplicación final completa. Por ambos motivos, la estimación se emplea como orientación del reparto estático/dinámico, y la caracterización efectiva se realiza midiendo directamente.

Medida directa sobre la plataforma

Para obtener el consumo real se midió directamente sobre la Arty S7-50, alimentada a 12 V por el pin Vin a través de una resistencia de derivación (*shunt*) de 1 Ω en serie con la línea de alimentación. La caída de tensión en la resistencia, medida con un multímetro, proporciona por la ley de Ohm la corriente demandada, y de ella la potencia total de la placa.

La medida se realizó a tres niveles, para diferenciar qué parte del consumo corresponde al sistema y qué parte al soporte de la propia placa. En primer lugar, se midió la placa completa en funcionamiento. A continuación, se midió la placa en reposo, sin programa cargado, que representa el consumo base de la plataforma. La diferencia entre ambas cifras aísla el consumo atribuible al sistema en ejecución. Por último, se emplearon los puntos de medida que la

placa expone en los conectores JP3 y JP4, que permiten registrar la corriente de los raíles VCCINT (1 V) y VCC1V8 (1,8 V) correspondiente al núcleo de la FPGA y su circuitería auxiliar, como se indica en [9]. La Tabla 6.9 recoge las medidas obtenidas.

Medida	Potencia Sist. Migrado (W)	Potencia Sist. Ampliado (W)
Placa completa en funcionamiento	$2,0671 \pm 0,103$	$2,332 \pm 0,116$
Placa completa en reposo	$0,9765 \pm 0,048$	$0,9765 \pm 0,048$
Sistema en ejecución (diferencia completa y reposo)	1,0906	1,3555
Raíles VCCINT + VCC1V8	$0,836 \pm 0,041$	$1,172 \pm 0,058$

Tabla 6.9 Consumo de potencia medido sobre la Arty S7-50 para los distintos sistemas. Margen de error atribuido al 5 % de tolerancia de la resistencia *shunt*.

Los resultados admiten varias lecturas. La medida de la placa completa engloba tanto la alimentación basal de la plataforma como el consumo del sistema en funcionamiento. La diferencia entre la completa y la medida en reposo constituye, por tanto, la referencia más representativa del coste real del sistema, al recoger el consumo necesario al ejecutarse sin el sobre coste fijo de la placa.

Sobre esa magnitud, el sistema básico se mantiene por debajo de los 1,1 W, una cifra reducida para un canal de vídeo en tiempo real, atribuible al procesamiento sobre hardware dedicado. La ampliación eleva ese consumo en torno a un 24 %, un incremento modesto pero notable que se corresponde con la lógica añadida en el Capítulo 5: el módulo de procesamiento HLS, que opera sobre cada píxel del flujo, y el subsistema de monitorización, activo de forma

continua. El crecimiento del consumo es, así, proporcional a la funcionalidad incorporada.

La medida por raíles permite, además, situar la estimación de Vivado en su contexto. Como la herramienta estima el consumo del chip y no de la placa, su contraste pertinente es con los raíles del propio dispositivo: a ese nivel, estimación y medida resultan del mismo orden, quedando la medida experimental algo por debajo al no abarcar estos raíles la totalidad del chip, ya que fuera de ellos permanecen las líneas que alimentan la actividad de E/S y el banco asociado a la memoria externa. Por tanto, la estimación es razonable en magnitud absoluta. Donde sí se queda corta es en reflejar el efecto de las ampliaciones. La diferencia de consumo entre ambos diseños medida sobre los raíles es de unos 350mW, superando los 20mW que estimaba Vivado, lo que evidencia su dificultad para caracterizar la actividad real del sistema.

6.6. Comparación con el sistema de referencia

Finalmente, resta comparar el sistema migrado con el proyecto original de partida. La comparación se realiza sobre el canal de vídeo básico, común a ambos, dejando fuera del análisis las ampliaciones del Capítulo 5, que no tienen equivalente en la referencia.

Potencia

La comparación de consumo se realiza sobre la potencia total de cada placa en funcionamiento, medida que ambas permiten obtener. En la Arty S7 se sigue el método de la sección anterior, mientras que la Zybo Z7 cuenta con un medidor integrado que monitoriza la corriente total que consume la placa, accesible desde la aplicación software mediante el bloque XADC (*Xilinx Analog-to-Digital Converter*). Siguiendo el procedimiento y la fórmula dadas por el fabricante en [16], se obtiene la corriente consumida. La Tabla 6.10 recoge los resultados.

Plataforma	Configuración	Potencia (W)
Zybo Z7	Nativa (667 MHz CPU, 533 MHz DDR, 150 MHz AXI)	$2,668 \pm 0,101$
Zybo Z7	Ajustada (100 MHz CPU, 325 MHz DDR, 80 MHz AXI)	$2,308 \pm 0,071$
Arty S7-50	Sistema migrado	$2,0671 \pm 0,103$

Tabla 6.10 Comparativa de potencia entre sistema original y migrado.

En la configuración nativa del sistema de referencia, la Zybo opera a frecuencias muy superiores a la del sistema migrado, lo que eleva su consumo. Para hacer una comparación justa entre soluciones y no implementaciones concretas, se han ajustado las frecuencias de operación de la Zybo para que se aproximen a la situación del sistema migrado.

Una vez igualadas las condiciones, ambas plataformas están un rango de consumo próximo, siendo la potencia de la Arty algo menor al sistema de referencia. La comparación debe matizarse: las medidas proceden de instrumentos distintos y cada placa arrastra su propio consumo base, distinto en una y otra. Por ello, el resultado debe leerse como una equivalencia en orden de magnitud más que como una diferencia exacta. La imposibilidad de medir directamente la potencia del chip o de la plataforma en reposo en el caso de la Zybo Z7 hace que esta sea la comparativa más representativa.

Aun con esa cautela, la conclusión es clara: reproducir el canal de vídeo sobre la FPGA discreta no conlleva una penalización energética. Al contrario, su consumo se mantiene en el mismo orden o por debajo del de la plataforma SoC de referencia operando en condiciones equivalentes.

Conviene contrastar estas medidas con la estimación de Vivado que, en el caso del dispositivo SoC, resulta especialmente ilustrativo. La Tabla 6.11 recoge las estimaciones de la herramienta para cada implementación.

Plataforma	Configuración	Potencia Estática (W)	Potencia Dinámica (W)	Potencia Total (W)
Zybo Z7	Nativa	0,129	1,822	1,951
Zybo Z7	Ajustada	0,122	1,455	1,577
Arty S7-50	Migrado	0,077	1,030	1,107

Tabla 6.11. Estimación de consumo de Vivado para el sistema de referencia y el migrado.

La estimación revela el peso que el Sistema de Procesamiento tiene en el consumo de un dispositivo SoC. Vivado atribuye al PS buena parte de la potencia dinámica de la Zybo, más de 1 W en ambas configuraciones, y al ajustar su configuración la estimación del consumo total desciende de forma apreciable. Esta reducción, en torno a 350 mW, se evidencia también en la medida real de la Tabla 6.10.

Conviene, por tanto, una lectura matizada: si bien Vivado parece sobrestimar la contribución del PS, como advierten otros trabajos [17], la tendencia de fondo es real. El consumo de un SoC está fuertemente condicionado por su procesador integrado, lo que lo sitúa por encima de una solución puramente discreta que prescinde de él. Esta diferencia estructural concuerda con lo observado experimentalmente, donde la Arty se mantiene por debajo de la Zybo en condiciones iguales, y aporta una explicación a esa ventaja: parte del menor consumo de la solución discreta proviene, precisamente, de no arrastrar el coste energético inherente al Sistema de Procesamiento. Este mayor consumo también se evidencia en la potencia estática, que es ligeramente superior en el caso de la Zybo.

Recursos

La comparación de área requiere una lectura cuidadosa, pues cada plataforma implementa la estructura del canal de forma diferente. La Tabla 6.12

recoge los recursos absolutos, mientras que la Figura 6.11 muestra el porcentaje de ocupación de cada placa.

Plataforma	LUT	FF	BRAM
Zybo Z7-10	6214	9056	7
Arty S7-50	12439	14655	23

Tabla 6.12 Ocupación de recursos del sistema en ambas plataformas.

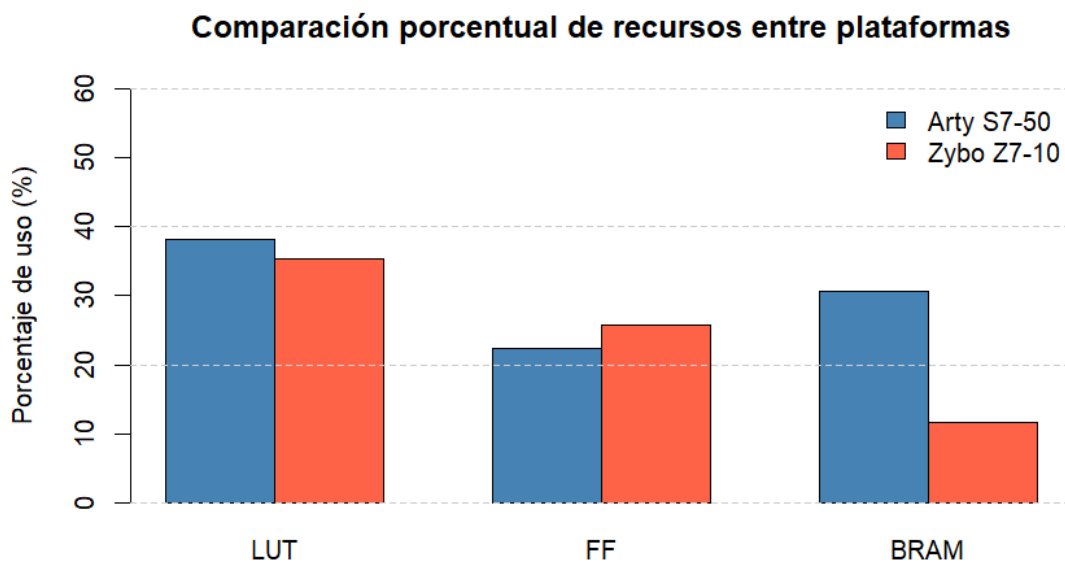


Figura 6.11 Ocupación de cada sistema respecto del total de cada plataforma.

El sistema migrado emplea cerca del doble de LUT que el proyecto de referencia. Lejos de ser una desventaja, esa diferencia cuantifica el coste de reproducir el controlador de memoria externa, el procesador y los periféricos que en la Zybo residen en el Sistema de Procesamiento. Es, en esencia, el coste en

lógica de la migración. El número de BRAM empleados manifiesta esto, pues la diferencia son los 16 empleados por MicroBlaze como memoria local.

Sin embargo, una mayor ocupación absoluta no significa una menor capacidad de crecimiento, sino al contrario. Aunque en la gráfica se observa que en ambas plataformas se ocupa en torno a un tercio de la PL, dos tercios representan más LUT libres en la Arty que en la Zybo, al ser un dispositivo de mayor densidad lógica. Por tanto, el sistema migrado dispone de un margen de ampliación sensiblemente superior al de referencia.

Las ampliaciones desarrolladas en el Capítulo 5 lo manifiestan: sobre la Arty elevaron la ocupación a un 51%. Ese mismo coste, en torno a 4200 LUT como se recoge en la Tabla 6.6, llevaría la ocupación en la Zybo Z7 a casi un 60%. En definitiva, la plataforma discreta no solo puede alojar el canal de vídeo, sino que ofrece un mayor margen de crecimiento.

Valoración

Para el canal de vídeo de este proyecto, ambas plataformas han mostrado la misma funcionalidad, un margen de recursos adecuado y un consumo de potencia del mismo orden. La diferencia decisiva entre ambas implementaciones es el coste. La recoge los recursos de lógica programable y el precio de varios dispositivos de las familias Spartan-7 y Zynq-7000.

Dispositivo	Familia	PS	LUT	FF	BRAM	DSP	Precio (€)	€/kLUT
XC7S25	Spartan-7	No	14600	29200	45	80	38,85	2,66
XC7S50	Spartan-7	No	32600	65200	75	120	64,91	1,99
XC7Z010	Zynq-7000	Sí	17600	35200	60	80	62,21	3,53
XC7Z015	Zynq-7000	Sí	46200	92400	95	160	133,65	2,89

Tabla 6.13 Recursos de lógica programable y coste de dispositivos discretos y SoC. Precios unitarios de Digikey sin IVA [18].

Los datos son reveladores. Tanto el chip XC7S50 de la Spartan-7 como el XC7Z010 del sistema de referencia se sitúan en torno al mismo precio, pero el dispositivo discreto ofrece casi el doble de lógica programable. En el caso del SoC, el coste se reparte entre los recursos lógicos y el Sistema de Procesamiento.

Esta tendencia se muestra en la última columna, que evidencia como el coste por unidad de lógica es menor en los dispositivos discretos. Esta métrica favorece notablemente a la opción discreta en las situaciones como en la que se sitúa este proyecto: cuando la aplicación puede prescindir del Sistema de Procesamiento.

Esta ventaja se proyecta además sobre el margen de crecimiento. El sistema migrado ocupa unas 12000 LUT, por lo que la Arty S7-50 conserva alrededor de 21000 LUT libres, frente a las 11600 que deja el canal de referencia en la Z010. La plataforma discreta posee el doble de lógica disponible y, además, más barata por unidad. El coste de escalar el sistema, como añadiendo las ampliaciones del Capítulo 5, resulta sensiblemente menor sobre la FPGA discreta, de modo que su ventaja económica no se limita al sistema actual, sino que se amplía a medida que este crece. Aunque si únicamente se quisiera desarrollar el canal de vídeo básico, el chip XC7S25 es la opción más económica para la que se ha probado la portabilidad.

Con todo, la viabilidad de una FPGA discreta para este sistema no se decide solo a nivel del chip. Un canal de vídeo completo necesita una placa que aporte los recursos físicos imprescindibles: memoria DDR conectada a la FPGA, conectores para la cámara y una salida de vídeo. Este trabajo demuestra que tal placa existe y es asequible: dentro del catálogo de Digilent accesible en [19], la Arty S7-50 resulta más económica que la Zybo Z7 de referencia y reúne lo necesario. La diferencia es que la Zybo integra un conector HDMI y la Arty no, por lo que la salida hubo de resolverse mediante la adaptación descrita en el apartado 4.5. Esa es la contrapartida de la opción más económica: un menor coste a cambio de prescindir de ciertas comodidades de integración, suplidas con un trabajo de adaptación asumible.

Para el sistema desarrollado, el balance se inclina hacia la plataforma discreta, que proporciona los recursos necesarios a menor precio sin renunciar a ninguna funcionalidad del canal de vídeo, como se ha demostrado.

6.7. Limitaciones

El canal de vídeo final implementado presenta ciertas limitaciones. En cuanto a la cámara, el sistema conoce su presencia porque el software la declara al activarla, no mediante detección física. Por ello, si el sensor se desconecta “en caliente” mientras opera un modo de procesado, el módulo queda a la espera de unos píxeles que no llegan y la cadena de vídeo se detiene hasta que el software reinicia la entrada. Una detección automática de la presencia del sensor evitaría este bloqueo.

La entrada y salida de vídeo presentan también una limitación menor. Las frecuencias de píxel de la cámara y del HDMI no coinciden de forma exacta con las nominales, lo que introduce la ligera desviación de la tasa de fotogramas cuantificada en el apartado 6.2.

Referido a la evaluación, las medidas de potencia para la Arty S7-50 y la Zybo se obtuvieron mediante instrumentos diferentes, por lo que los resultados deben interpretarse como una equivalencia de orden de magnitud y no como una diferencia precisa. Por último, tanto la plataforma de referencia como la migrada pertenecen al ecosistema de AMD/Xilinx y el desarrollo se apoya en sus herramientas y su catálogo de IP, lo que condiciona la portabilidad de las soluciones a FPGA de otros fabricantes.

Capítulo 7. Conclusiones

La cuestión principal de este trabajo era si un canal de vídeo en tiempo real concebido sobre una plataforma SoC podía trasladarse a una FPGA discreta y, de ser así, a qué coste y con qué rendimiento. La respuesta es afirmativa, y el sistema construido la respalda con medidas reales.

Reproducir sobre lógica programable el procesador, el controlador de memoria y los periféricos que antes resolvía el Sistema de Procesamiento tiene un coste asumible y un resultado satisfactorio: la aplicación de control se mantuvo prácticamente intacta y el vídeo se transmite sin pérdidas. El sistema no solo iguala las funcionalidades de la referencia, sino que conserva un amplio margen de crecimiento, aprovechado para incorporar funciones nuevas que pusieron a prueba la interacción hardware-software. En prestaciones, el sistema migrado se iguala al de partida.

De ese margen nace el verdadero hallazgo. Demostrado que el Sistema de Procesamiento puede llevarse a lógica sin penalización apreciable, la elección entre un SoC y una FPGA discreta deja de ser técnica para volverse económica: por un precio comparable, la FPGA discreta ofrece mayor densidad de lógica programable que un SoC de gama equivalente, donde parte del coste se destina a recursos integrados que no toda aplicación aprovecha. Cuando el diseño puede prescindir del procesador dedicado, migrar sus funciones a lógica no es una solución de compromiso, sino una alternativa competitiva y a menudo más rentable.

El trabajo concluye, así, con la idea que lo ha vertebrado: con las decisiones de diseño correctas y un dimensionado ajustado a los requisitos, una FPGA discreta puede albergar sistemas completos, con interfaces, accesos a memoria y control software, que tradicionalmente habrían recurrido a la comodidad de un procesador integrado.

7.1. Líneas futuras

El proyecto plantea distintas líneas de trabajo.

Se pueden explorar otras opciones de procesadores *soft-core*, como PicoBlaze, para dotar al canal de vídeo de una unidad de control aún más ajustada a la complejidad de sus tareas.

Emplear cámaras o salidas de visualización de mayor resolución o con formatos de color más exigentes, lo que implicaría reevaluar la capacidad de los accesos a memoria externa y las frecuencias de operación.

Confirmar la portabilidad del canal de vídeo básico a una FPGA XC7S25, que la implementación de Vivado apunta como alcanzable. Digilent ofrece una plataforma Arty S7-25 similar a la empleada en este proyecto, pero incluyendo el chip citado. Se podría incluso estudiar un dimensionamiento aún más ajustado del sistema que permitiera añadir alguna funcionalidad extra.

Ampliar las capacidades del Plano de Datos con algoritmos de procesamiento más complejos en HLS.

Para explorar aún más la capacidad de una FPGA discreta, se puede estudiar la implementación de dos sistemas independientes en un mismo chip, cada uno gobernado por un *soft-core* distinto.

Referencias

- [1] Xilinx, “7 Series FPGAs Data Sheet: Overview DS180 (v2.6.1)”, septiembre, 2020. Accesible en: https://docs.amd.com/v/u/en-US/ds180_7Series_Overview (Consultado el 13 de junio de 2026)
- [2] Intel, "Pipelining, FPGA Optimization Guide for Intel oneAPI Toolkits" oct., 2023. Accesible en: <https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/optimization-guide/2023-1/pipelining-001.html> (Consultado el 16 de mayo de 2026).
- [3] Xilinx, “Vivado Design Suite: AXI Reference Guide (UG1037)”, julio, 2017. Accesible en: <https://docs.amd.com/v/u/en-US/ug1037-vivado-axi-reference-guide> (Consultado el 30 de mayo de 2026)
- [4] Applied Logix, J. Powers, “Choosing Between FPGA, Processor, and SoC” Applied Logix Blog, junio, 2025. Accesible en: <https://appliedlogix.rebuildmanufacturing.com/blogs/choosing-between-fpga-processor-and-soc> (Consultado el 30 de mayo de 2026)
- [5] AMD, “Vivado Design Suite: MicroBlaze Processor Reference Guide (UG984),” noviembre, 2025. Disponible en: <https://docs.amd.com/r/en-US/ug984-vivado-microblaze-ref/> (Consultado el 31 de mayo de 2026)
- [6] A. F. Mondragón y J. Christman “Hard Core vs. Soft Core: A Debate,” en Rochester Institute of Technology, Nueva York, EE UU, 2012.
- [7] Xilinx, “Zynq-7000 SoC Data Sheet: Overview”, julio, 2018. Accesible en: <https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview> (Consultado el 29 de mayo de 2026)

- [8] AMD, "AXI Video Direct Memory Access v6.3 Product Guide (PG020)", junio, 2022. Accesible en: https://docs.amd.com/r/en-US/pg020_axi_vdma (Consultado el 19 de mayo de 2026)
- [9] Digilent, "Arty S7 Reference Manual," 2024. Disponible en: <https://digilent.com/reference/programmable-logic/arty-s7/reference-manual> (Consultado el 14 de junio de 2026)
- [10] AMD, "7 Series FPGAs Memory Interface Solutions v1.7 User Guide (AXI)" octubre, 2012. Accesible en: https://docs.amd.com/v/u/1.7-English/ug586_7Series_MIS (Consultado el 2 de junio de 2026)
- [11] AMD, "SmartConnect (PG247)" octubre, 2022. Accesible en: <https://docs.amd.com/r/en-US/pg247-smartconnect> (Consultado el 10 de junio de 2026)
- [12] AMD, "AXI Interconnect LogiCORE IP Product Guide (PG059)" <https://docs.amd.com/r/en-US/pg059-axi-interconnect> (Consultado el 12 de junio de 2026)
- [13] Semtech, A. Bhattacharya. "ESD Protection for HDMI 2.0" Semtech Blog, octubre, 2020. Accesible en: <https://blog.semtech.com/esd-protection-for-hdmi-2.0> (Consultado el 23 de mayo de 2026)
- [14] AMD, "LogiCORE IP AXI Performance Monitor v1.00.a Product Guide (AXI)" mayo, 2012. Accesible en: https://docs.amd.com/v/u/en-US/pg037_axi_perf_mon (Consultado el 10 de junio de 2026)
- [15] AMD, "AXI Timer 2.0 LogiCORE IP Product Guide (PG079)" agosto, 2025. Accesible en: <https://docs.amd.com/r/en-US/pg079-axi-timer> (Consultado el 10 de junio de 2026)
- [16] Digilent, "Zybo Z7 Reference Manual" Accesible en: <https://digilent.com/reference/programmable-logic/zybo-z7/reference-manual> (Consultado el 14 de junio de 2026)
- [17] V. J. Vazhoth Kanhiroth, ""Embedded processors on FPGA: Hard-core vs Soft-core" en Grand Valley State University, Míchigan, EE UU, 2017.
- [18] Digikey, "Products" Accesible en: <https://www.digikey.es/> (Consultado el 20 de junio de 2026)

- [19] Digilent, “FPGA Boards”. Accesible en:
https://digilent.com/shop/products/fpga-boards/?srsltid=AfmBOorTv2noxQY_3qQpbOtgL_5hpfSiuoZ8HEtfUNHQSTkCI5dMCJ0e (Consultado el 15 de junio de 2026)

Apéndice A. Software adjunto y repositorio del proyecto

El material desarrollado durante este Trabajo de Fin de Grado (el diseño hardware en Vivado, la aplicación software de control, el código del módulo de procesamiento diseñado mediante HLS y la presente memoria) se encuentra disponible en un repositorio público accesible en la siguiente dirección:

[AlfonsoSuarezGasull/TFG-migracion-video-fpga:](https://github.com/AlfonsoSuarezGasull/TFG-migracion-video-fpga)

El código fuente íntegro se recoge en su totalidad en dicho repositorio. En el cuerpo del documento se han incluido únicamente los fragmentos puntuales necesarios para ilustrar decisiones de diseño concretas.

El repositorio se organiza en los siguientes directorios. El directorio *codigo/hls/* contiene el código C++ del módulo de procesamiento y generación de vídeo descrito en el apartado 5.3. El directorio *codigo/software/* recoge la aplicación de control *bare-metal* ejecutada por el procesador MicroBlaze, cuyo funcionamiento se detalla en el apartado 5.5. El directorio *codigo/vivado/* incluye el script Tcl que reconstruye el proyecto completo de Vivado, junto con los archivos que este requiere: el archivo de restricciones de la placa, el *wrapper* del diseño, la configuración del controlador de memoria (MIG) y los bloques IP externos empleados. Finalmente, el directorio *memoria/* contiene esta memoria en formato PDF y el directorio *presentacion/* la presentación empleada en la defensa.

El repositorio incluye un fichero *README.txt* con la descripción detallada de su contenido y las instrucciones completas para el montaje del hardware, la reconstrucción del proyecto en Vivado y la programación de la placa.

